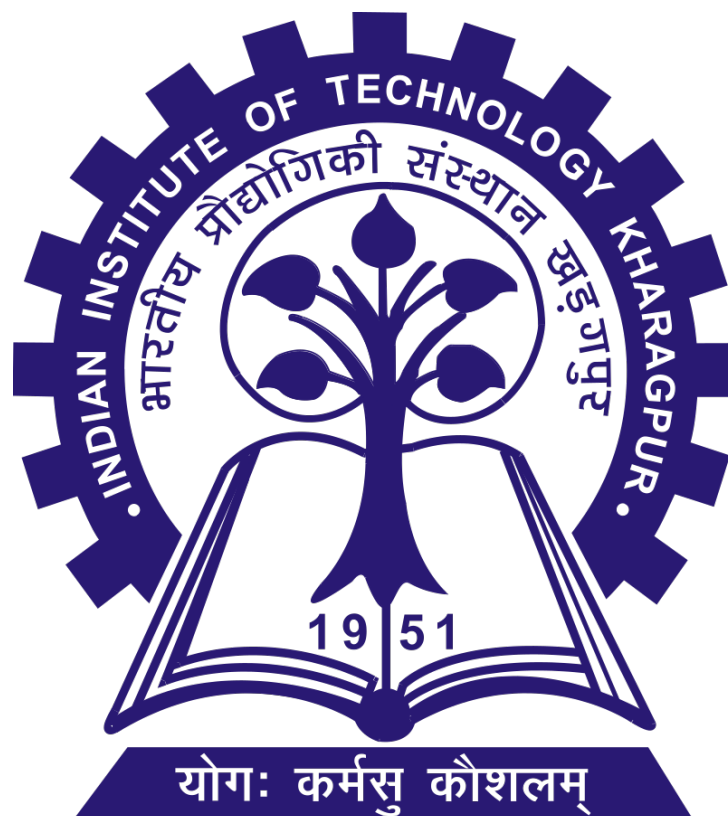


Digital Signal Processing Laboratory (EC39201) IIT Kharagpur

Experiment 2: FIR and IIR Filters

Name: Sanjay R, Rishant Mallick
Roll No: 23EC10101, 23EC10067
Group No: 3
Date of submission: 9-10-2025



Theory

Window Method for FIR Filter Design

The window method for digital filter design is fast, convenient, and robust, but generally suboptimal. The basic idea is to truncate a theoretically ideal impulse response $h_d(n)$ using a suitably chosen window function $w(n)$, yielding:

$$h(n) = h_d(n) \cdot w(n)$$

Time Limitation

Window functions are always time-limited. This means there exists a finite integer N_w such that

$$w(n) = 0 \quad \text{for all } |n| > N_w$$

Thus, the final windowed impulse response $h(n)$ is always time-limited, as required for practical implementation. Therefore, the window method always produces a finite impulse response (FIR) digital filter, as opposed to an infinite impulse response (IIR) digital filter.

Frequency Domain Interpretation

By the dual of the convolution theorem, pointwise multiplication in the time domain corresponds to convolution in the frequency domain. Hence, the designed filter $h(n)$ has a frequency response:

$$H(\omega) = \frac{1}{2\pi} \int_{-\pi}^{\pi} H_d(\nu) W(\omega - \nu) d\nu$$

where $H_d(\omega) = \text{DTFT}\{h_d(n)\}$ is the ideal frequency response, and $W(\omega) = \text{DTFT}\{w(n)\}$ is the window transform.

Ideal Low-Pass Filter

For the ideal low-pass filter, $H_d(f)$ is a rectangular function in the frequency domain. Therefore, the practical frequency response $H(f)$ is obtained by convolving this rectangular function with the window transform $W(f)$.

Pass-Band and Stop-Band Behavior

- The **pass-band gain** is primarily determined by the area under the main lobe of the window transform, provided that the main lobe fits within the pass-band (i.e., the total low-pass bandwidth is greater than or equal to the main-lobe width of W).
- The **stop-band gain** is governed by an integral over the side lobes of the window transform. Since side-lobes oscillate about zero, their integral tends to be much smaller than their peak values due to cancellation between adjacent lobes.

When designing an FIR filter:

- The ideal filter has an infinite-length impulse response (e.g., ideal lowpass is a sinc function).

- In practice, we truncate it, making it finite.
- Truncation corresponds to multiplying by a window function in the time domain.
- By the convolution theorem, this becomes a convolution in the frequency domain, smearing the ideal frequency response with the window's spectrum.

Thus, the choice of window is crucial:

- It determines the **main-lobe width** (transition band sharpness).
- It determines the **side-lobe level** (stopband attenuation).
- Always a tradeoff: a narrow main lobe vs. low side lobes.

1. Rectangular Window

Formula:

$$w(n) = \begin{cases} 1, & 0 \leq n \leq N - 1 \\ 0, & \text{otherwise} \end{cases}$$

Interpretation: The rectangular window simply truncates the ideal infinite impulse response to a finite length. This abrupt cutoff in time corresponds to a convolution with a sinc-like spectrum in frequency, which produces strong ripples (Gibbs phenomenon). The benefit is a very narrow main lobe, meaning sharp frequency resolution, but the side-lobes are very high (only about -13 dB down). Thus, it provides excellent resolution but very poor attenuation, making it unsuitable for practical filtering tasks where leakage matters.

Frequency Domain:

- Main lobe width: $\frac{4\pi}{N}$
- First side-lobe: ≈ -13 dB
- Side-lobe decay: $1/\omega$

Advantages:

- Sharpest frequency resolution.
- Computationally trivial.

Disadvantages:

- Very high spectral leakage.
- Weak stop-band attenuation.

2. Triangular (Bartlett) Window

Formula:

$$w(n) = \begin{cases} 1 - \frac{2|n - \frac{N-1}{2}|}{N-1}, & 0 \leq n \leq N-1 \\ 0, & \text{otherwise} \end{cases}$$

Interpretation: The triangular window linearly tapers to zero at the edges, smoothing the discontinuities present in the rectangular case. This tapering lowers the side-lobe levels (about -26 dB), reducing leakage, but also widens the main lobe, reducing frequency resolution. It is therefore a compromise — better suppression than rectangular, but less sharp in distinguishing nearby frequencies.

Frequency Domain:

- Main lobe width: $\frac{8\pi}{N}$
- First side-lobe: ≈ -26 dB
- Side-lobe decay: $1/\omega^2$

Advantages:

- Better suppression than rectangular.
- Simple implementation.

Disadvantages:

- Wider main lobe $\Rightarrow$ poorer resolution.

3. Hanning (Hann) Window

Formula:

$$w(n) = \begin{cases} 0.5 - 0.5 \cos\left(\frac{2\pi n}{N-1}\right), & 0 \leq n \leq N-1 \\ 0, & \text{otherwise} \end{cases}$$

Interpretation: The Hanning window uses a cosine taper that smoothly reaches zero at both ends. This smoothness significantly reduces spectral leakage compared to triangular, with side-lobes suppressed to around -31 dB and decaying faster. However, the main lobe is somewhat wide, meaning reduced resolution. It is widely used in spectral analysis where a good balance between leakage suppression and resolution is needed.

Frequency Domain:

- Main lobe width: $\frac{8\pi}{N}$
- First side-lobe: ≈ -31 dB
- Side-lobe decay: $1/\omega^3$

Advantages:

- Better stop-band attenuation than triangular.

- Smooth roll-off, balanced compromise.
- Widely used in spectral analysis.

Disadvantages:

- Wider main lobe than rectangular.
- Stop-band suppression weaker than Hamming/Blackman.

4. Hamming Window

Formula:

$$w(n) = \begin{cases} 0.54 - 0.46 \cos\left(\frac{2\pi n}{N-1}\right), & 0 \leq n \leq N-1 \\ 0, & \text{otherwise} \end{cases}$$

Interpretation: The Hamming window is similar to the Hanning window but modified so that the edges do not fall to zero, instead ending near 0.08. This small adjustment forces the first side-lobe to be much lower (about -41 dB), making it very effective for filtering applications. The trade-off is that the decay rate of side-lobes is slower than Hann or Blackman. In practice, it is one of the most popular choices for FIR filter design, as it combines reasonable resolution with excellent leakage suppression.

Frequency Domain:

- Main lobe width: $\frac{8\pi}{N}$
- First side-lobe: ≈ -41 dB
- Side-lobe decay: $1/\omega^2$

Advantages:

- Strong side-lobe suppression.
- Very good choice for FIR filter design.
- Reduces leakage significantly.

Disadvantages:

- Main lobe wider than rectangular.
- Side-lobe decay slower than Hann or Blackman.

5. Blackman Window

Formula:

$$w(n) = \begin{cases} 0.42 - 0.5 \cos\left(\frac{2\pi n}{N-1}\right) + 0.08 \cos\left(\frac{4\pi n}{N-1}\right), & 0 \leq n \leq N-1 \\ 0, & \text{otherwise} \end{cases}$$

Interpretation: The Blackman window uses two cosine terms to create a very smooth taper. This produces excellent suppression of side-lobes (around -58 dB) and rapid

decay, minimizing leakage almost completely. However, the main lobe is the widest among the common windows, meaning frequency resolution is sacrificed. It is best suited to applications where strong stop-band attenuation is more important than distinguishing closely spaced frequencies.

Frequency Domain:

- Main lobe width: $\frac{12\pi}{N}$
- First side-lobe: ≈ -58 dB
- Side-lobe decay: $1/\omega^3$

Advantages:

- Excellent stop-band attenuation.
- Very low leakage (clean spectrum).

Disadvantages:

- Very wide main lobe (lowest resolution).
- Computationally more expensive.

6. Kaiser Window

Formula:

$$w(n) = \begin{cases} \frac{I_0\left(\beta\sqrt{1-\left(\frac{2n}{N-1}-1\right)^2}\right)}{I_0(\beta)}, & 0 \leq n \leq N-1 \\ 0, & \text{otherwise} \end{cases}$$

where $I_0(\cdot)$ is the zeroth-order modified Bessel function of the first kind, and β is a parameter controlling the window shape.

Interpretation: The Kaiser window uses a Bessel function shape to provide a flexible trade-off between main lobe width and side-lobe level through the parameter β . This allows near-optimal energy concentration in the frequency domain, providing strong stop-band attenuation with manageable main lobe width.

Frequency Domain:

- Main lobe width increases with increasing β .
- First side-lobe level typically ranges between -60 dB to -80 dB depending on β .
- Side-lobe decay improves (decays faster) as β increases.

Advantages:

- Adjustable trade-off between frequency resolution and side-lobe attenuation via β .
- Excellent stop-band attenuation.
- Low spectral leakage for appropriate choices of β .

Disadvantages:

- Computational cost due to Bessel function calculation.
- Wider main lobe at high β , reducing frequency resolution.

Calculation of β and A

The Kaiser window shape is controlled primarily by the parameter β , which depends on the desired stop-band attenuation A (in dB).

$$\beta = \begin{cases} 0, & A \leq 21 \\ 0.5842(A - 21)^{0.4} + 0.07886(A - 21), & 21 < A \leq 50 \\ 0.1102(A - 8.7), & A > 50 \end{cases}$$

Interpretation: - For $A \leq 21$ dB, the Kaiser window reduces to a rectangular window ($\beta = 0$). - Higher A values yield larger β , increasing side-lobe suppression at the expense of main lobe width. .

Summary Table

Window	Main Lobe Width	First Side-lobe (dB)	Decay Rate	Notes
Rectangular	$4\pi/N$	-13	$1/\omega$	Sharpest, poor attenuation
Triangular	$8\pi/N$	-26	$1/\omega^2$	Cleaner, less sharp
Hanning	$8\pi/N$	-31	$1/\omega^3$	Smooth, balanced
Hamming	$8\pi/N$	-41	$1/\omega^2$	Good suppression
Blackman	$12\pi/N$	-58	$1/\omega^3$	Very clean, wide band

Part A:

Theory

Noise, Types of Noise, and SNR

1. What is Noise?

Noise is any unwanted or random disturbance that affects a signal in a system. It can distort the signal or make it harder to extract meaningful information. Sources of noise include:

- Thermal agitation in electronic components (thermal noise).
- Quantization errors in digital systems (quantization noise).
- External electromagnetic interference (EMI).
- Random fluctuations in measurements (white noise).

Intuition: Noise is like static on a radio—it interferes with the true signal.

2. Types of Noise Added in Systems

In simulations or experiments, noise is often intentionally added to test system robustness. Common types include:

1. Additive White Gaussian Noise (AWGN):

- Zero mean, constant power across all frequencies.
 - Represents thermal noise and is mathematically convenient.
2. **Impulse Noise:** Sudden spikes of high amplitude, modeling burst errors.
 3. **Colored Noise (Pink, Brown):** Power is frequency-dependent.
 - Pink noise: power decreases as $1/f$.
 - Brown noise: power decreases as $1/f^2$.

Note: In most FIR/filter experiments, AWGN is commonly used.

3. Signal-to-Noise Ratio (SNR)

SNR measures how strong the signal is compared to noise.

$$\text{SNR (linear)} = \frac{P_{\text{signal}}}{P_{\text{noise}}}$$

In decibels (dB):

$$\text{SNR (dB)} = 10 \cdot \log_{10} \left(\frac{P_{\text{signal}}}{P_{\text{noise}}} \right)$$

Interpretation:

- Higher SNR $\rightarrow$ signal dominates noise (cleaner system)
- Lower SNR $\rightarrow$ noise dominates (signal hard to detect)

Intuition: SNR is like comparing the height of a voice to background chatter: a loud voice in a quiet room has high SNR; a whisper in a crowded place has low SNR.

4. Calculation of SNR

For a discrete-time signal $x[n]$ with noise $n[n]$, SNR can be calculated as follows:

1. Compute signal power:

$$P_{\text{signal}} = \frac{1}{N} \sum_{n=0}^{N-1} |x[n]|^2$$

2. Compute noise power:

$$P_{\text{noise}} = \frac{1}{N} \sum_{n=0}^{N-1} |n[n]|^2$$

3. Compute SNR:

$$\text{SNR (linear)} = \frac{P_{\text{signal}}}{P_{\text{noise}}}, \quad \text{SNR (dB)} = 10 \log_{10} \frac{P_{\text{signal}}}{P_{\text{noise}}}$$

Summary Table

Concept	Definition / Idea
Noise	Unwanted random disturbance affecting a signal
Common noise types	AWGN, Impulse, Colored (Pink, Brown)
SNR	Ratio of signal power to noise power; measure of signal quality
SNR calculation	Compute powers of signal and noise; $\text{SNR} = 10 \log_{10}(P_{\text{signal}}/P_{\text{noise}})$

Design of a Finite Impulse Response (FIR) low-pass filter using the window method.

A Finite Impulse Response (FIR) filter is a type of digital filter that has a finite-duration impulse response. FIR filters are widely used because they are always stable and can be designed to have linear phase, meaning the phase distortion is minimal and the waveform of the input signal is preserved in the output.

1. Filter Length and Symmetry

The number of filter taps is denoted by N , and the center of the filter is calculated as:

$$k = \frac{N - 1}{2}.$$

Symmetry around the center index k ensures that the filter has a **linear phase**, meaning that all frequency components are delayed equally. Linear phase is important because it preserves the shape of the input waveform in the passband.

2. Window Function

To implement a practical FIR filter, the infinite-length ideal impulse response must be truncated. This is done using a window function $w(n)$. Common window types include:

- **Rectangular:** simplest, sharp transition but high ripples in the stopband.
- **Triangular:** reduces ripples moderately.
- **Hanning/Hamming:** smooth taper, lower ripples, slightly wider transition.
- **Blackman:** very low side-lobe levels, but wider main lobe.

The window controls the trade-off between the **main-lobe width** (transition sharpness) and **side-lobe level** (stopband ripple).

3. Ideal Impulse Response

The ideal low-pass filter has an infinite-duration impulse response given by a **sinc function**:

$$h_d[n] = \begin{cases} \frac{\sin(W_c(n - k))}{\pi(n - k)}, & n \neq k \\ \frac{W_c}{\pi}, & n = k \end{cases}$$

where W_c is the **cutoff frequency** in radians/sample.

Symmetry of $h_d[n]$ about the center ensures linear phase. The sinc function represents a perfect low-pass filter in theory, passing all frequencies below W_c and rejecting higher frequencies.

4. Windowing the Impulse Response

The practical FIR filter coefficients are obtained by multiplying the ideal response by the chosen window:

$$h[n] = w[n] \cdot h_d[n].$$

This truncation introduces smoothing in the frequency domain, reducing the Gibbs phenomenon and resulting in a realizable filter.

5. Frequency Response

The frequency response of the filter is obtained using:

$$\text{freqz}(h),$$

which plots both the magnitude and phase response. This allows verification that the designed filter behaves as a low-pass filter and preserves linear phase.

6. Key Theory and Intuition

- FIR filters are inherently stable and have linear phase if their coefficients are symmetric.
- Windowing is a practical method to convert an ideal, infinite-length filter into a realizable FIR filter.
- The choice of window affects the balance between transition sharpness and ripple in the frequency response.
- Increasing the filter length N improves frequency selectivity by narrowing the transition band.

Role of the Window Function

The window method is one of the most widely taught and applied techniques for FIR filter design due to its simplicity, numerical stability, and straightforward implementation. The essential idea is to truncate the theoretically infinite impulse response of an ideal filter (such as the sinc function for a low-pass filter) using a finite-length window function. This ensures realizability of the filter in practice, since only a finite number of coefficients can be implemented in hardware or software. The choice of window function $w(n)$ plays a central role in determining the frequency-domain characteristics of the designed filter. Specifically:

- The **main-lobe width** of the window transform $W(\omega)$ dictates the **transition bandwidth** of the filter. A narrower main lobe yields sharper transitions between pass-band and stop-band.

- The **side-lobe levels** of $W(\omega)$ determine the **stop-band attenuation**. Lower side-lobes reduce leakage into the stop-band, improving the filter's ability to suppress unwanted frequencies.

Different window functions offer different trade-offs:

- The **Rectangular window** has the narrowest main lobe, but very high side-lobes (about -13 dB), leading to significant spectral leakage.
- The **Hanning and Hamming windows** increase the main-lobe width slightly but achieve much lower side-lobes (-31 dB for Hanning, -43 dB for Hamming), improving stop-band attenuation.
- The **Blackman window** offers excellent side-lobe suppression (around -58 dB), but at the cost of a very wide main lobe, resulting in a poor transition region.

Thus, the choice of window depends on whether sharp transition or strong stop-band attenuation is prioritized in the application.

Effect of Filter Length

The filter length N (or number of taps) also has a profound impact on performance:

- Increasing N reduces the transition bandwidth, since the main lobe of $W(\omega)$ narrows with increasing window length.
- Longer filters also lead to improved stop-band attenuation, since the frequency resolution of the design improves.
- However, longer filters increase computational cost (more multiplications per input sample) and delay (group delay is proportional to $N/2$).

Frequency-Domain Interpretation

By the convolution theorem, multiplying the infinite impulse response by the window in the time domain corresponds to convolving the ideal frequency response $H_d(\omega)$ with the window transform $W(\omega)$ in the frequency domain:

$$H(\omega) = H_d(\omega) * W(\omega).$$

This explains why the sharp rectangular spectrum of the ideal low-pass filter becomes “smeared” after windowing: the convolution spreads the sharp edges, creating a transition band, and introduces ripples in both pass-band and stop-band.

Advantages

- **Simplicity:** The method is conceptually easy and requires no iterative optimization.
- **Stability:** FIR filters are always stable, and symmetric designs guarantee linear phase.

- **Flexibility:** A wide range of filter types (low-pass, high-pass, band-pass, etc.) can be designed by suitable modulation and shifting.
- **Computational efficiency:** The design process is fast and suitable for educational and practical prototyping.

Limitations

Despite its usefulness, the window method is not optimal in a strict sense:

- The resulting filter does not minimize any formal error criterion (such as least-squares or Chebyshev error).
- Trade-offs between main-lobe width and side-lobe suppression are fixed by the choice of window; the designer has limited control.
- Transition bandwidth is relatively wide compared to more advanced methods such as Parks–McClellan (equiripple) design.

Code

```
clc;
clear;
close all;

number_of_samples = 64;
low_pass_filter = zeros(1,number_of_samples);
cutoff_frequency = 0.5*pi;

k = (number_of_samples-1)/2;

for n = 1:number_of_samples
    if n == k
        low_pass_filter(n) = cutoff_frequency/pi;
    else
        low_pass_filter(n) = (sin(cutoff_frequency*(n-k)))/(pi*(n-k));
    end
end

rectangular_window = ones(1,number_of_samples);
triangular_window = zeros(1,number_of_samples);
hanning_window = zeros(1,number_of_samples);
hamming_window = zeros(1,number_of_samples);
blackman_window = zeros(1,number_of_samples);

for n = 1:number_of_samples
    triangular_window(n) = 1 - 2*(n-(number_of_samples-1)/2)/(number_of_samples-1);
    hanning_window(n) = 0.5 - 0.5*cos(2*pi*n/(number_of_samples-1));
    hamming_window(n) = 0.54 - 0.46*cos(2*pi*n/(number_of_samples-1));
    blackman_window(n) = 0.42 - 0.5*cos(2*pi*n/(number_of_samples-1)) + 0.08*cos(4*pi*n/(number_of_samples-1));
end
```

Figure 1: Code Part 1

```

window_list = {rectangular_window,triangular_window,hanning_window,hamming_window,blackman_window};
window_titles = ["Rectangular Window","Triangular Window","Hanning Window","Hamming Window","Blackman Window"];

for i = 1:length(window_list)
    modified_filter = low_pass_filter.*window_list{i};
    [frequency_response,frequency_axis] = freqz(modified_filter,1,number_of_samples);
    figure(i);
    subplot(2,1,1);
    plot(frequency_axis,20*log10(abs(frequency_response)));
    title(strcat("Amplitude Response of ",window_titles(i)));
    xlabel("Normalised Frequency");
    ylabel("Amplitude in decibels");
    grid on;
    subplot(2,1,2);
    plot(frequency_axis,angle(frequency_response));
    title(strcat("Phase Response of ",window_titles(i)));
    xlabel("Normalised Frequency");
    ylabel("Phase in Radians");
end

|

sampling_frequency = 24e3;
f1 = 2e3;
f2 = 10e3;

time_axis = linspace(0,(number_of_samples-1)/sampling_frequency,number_of_samples);
input_signal = cos(2*pi*f1*time_axis) + 2*cos(2*pi*f2*time_axis);
noise = randn(1,number_of_samples)*0.2;

```

Figure 2: Code Part 2

```

fft_frequency_axis = linspace(-sampling_frequency/2,sampling_frequency/2,number_of_samples);

for i = 1:length(window_list)
    modified_filter = low_pass_filter.*window_list{i};
    input_signal_fft = fft(input_signal);
    filter_fft = fft(modified_filter);

    output_fft = input_signal_fft.*filter_fft;
    figure(i+5);
    grid on;
    plot(fft_frequency_axis,abs(fftshift(output_fft)));
    xlabel("Frequency Axis");
    ylabel("Amplitude");
    title(strcat("FFT Plot of filter using ",window_titles(i)," without noise"));
end

for i = 1:length(window_list)
    modified_filter = low_pass_filter.*window_list{i};
    input_signal_fft = fft(input_signal+noise);
    filter_fft = fft(modified_filter);

    output_fft = input_signal_fft.*filter_fft;
    figure(i+10);
    grid on;
    plot(fft_frequency_axis,abs(fftshift(output_fft)));
    xlabel("Frequency Axis");
    ylabel("Amplitude");
    title(strcat("FFT Plot of filter using ",window_titles(i)," with noise"));
end

```

Figure 3: Code Part 3

```

%Plotting SNR For each of the Windows
SNR_Array = zeros(1,length(window_list));
for i = 1:length(window_list)
    modified_filter = low_pass_filter.*window_list{i};
    input_signal_fft = fft(input_signal);
    input_signal_with_noise = fft(input_signal+noise);
    output_signal = ifft(input_signal_fft.*modified_filter);
    output_signal_with_noise = ifft(input_signal_with_noise.*modified_filter);
    noisy_output = output_signal_with_noise-output_signal;
    SNR_Array(i) = snr(output_signal,noisy_output);
end

figure(19);
barh(window_titles,SNR_Array);
xlabel("Window Name");
ylabel("SNR");
title("SNR of Various Windows");

```

Figure 4: Code Part 4

Plots

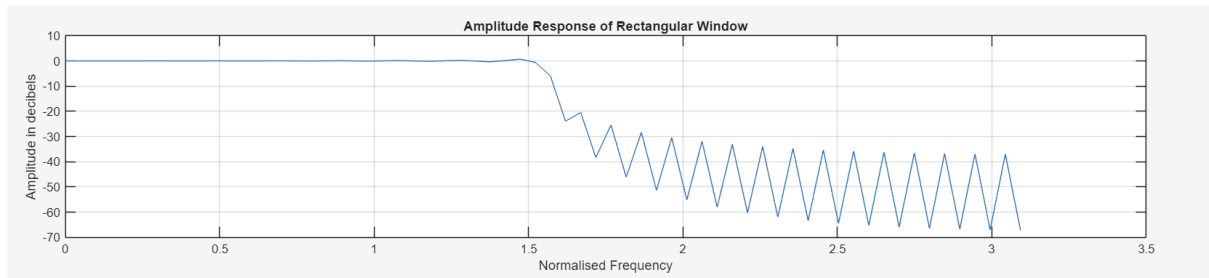


Figure 5: Amplitude Response for Rectangular Window

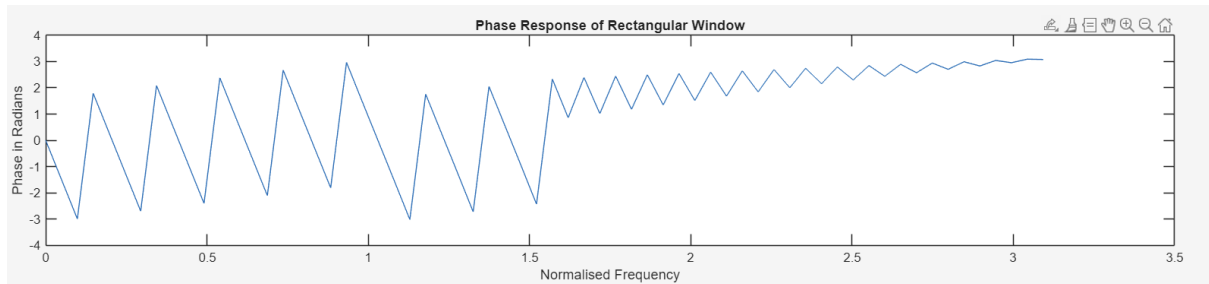


Figure 6: Phase Response for Rectangular Window

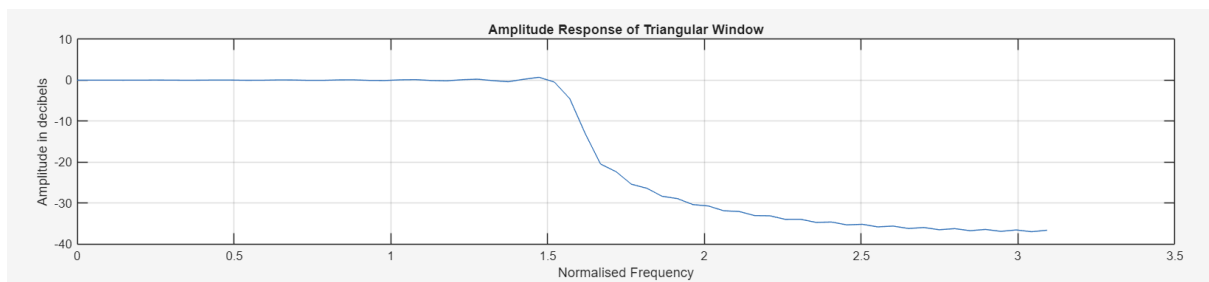


Figure 7: Amplitude Response for Triangular Window

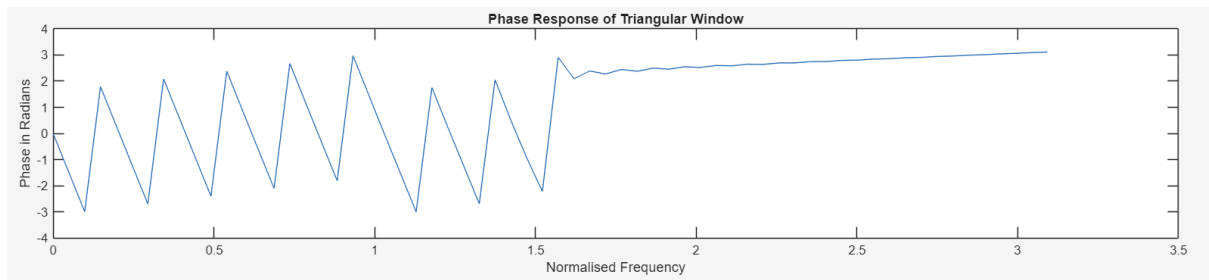


Figure 8: Phase Response for Triangular Window

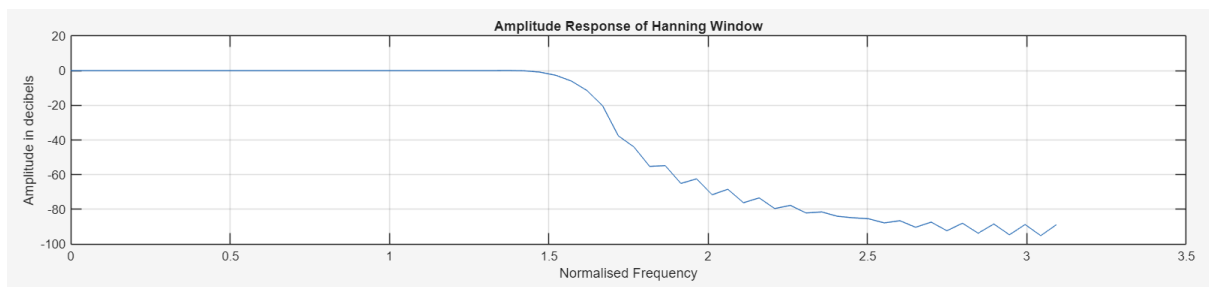


Figure 9: Amplitude Response for Hanning Window

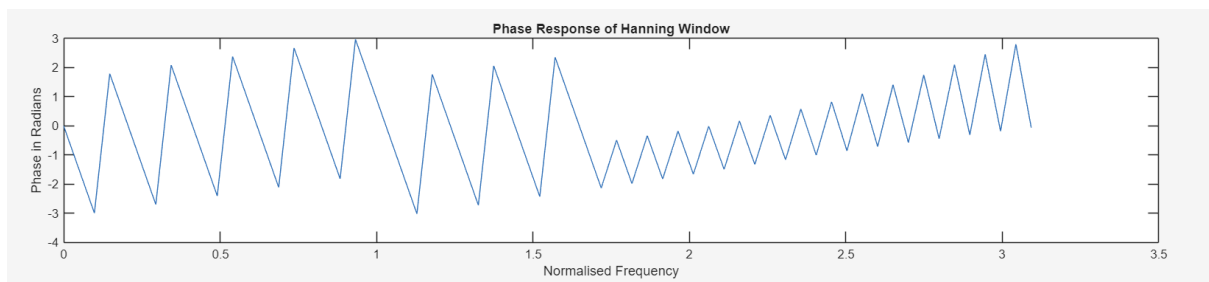


Figure 10: Phase Response for Hanning Window

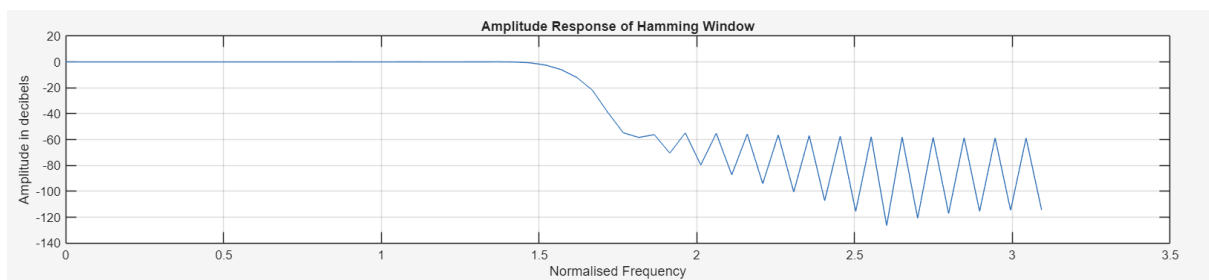


Figure 11: Amplitude Response for Hamming Window

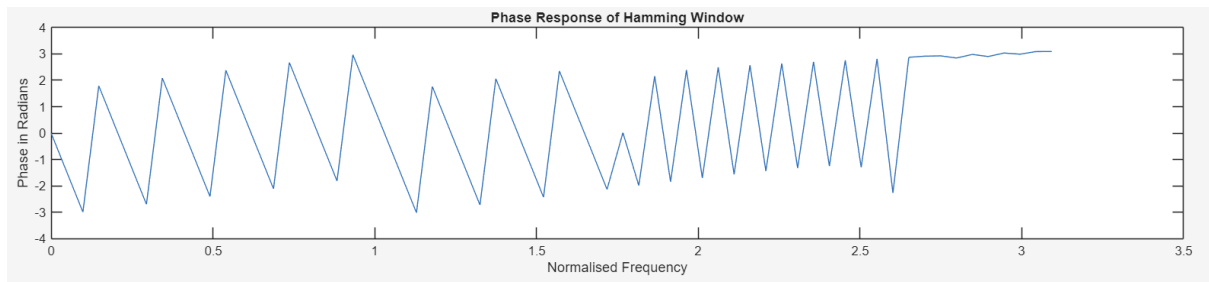


Figure 12: Phase Response for Hamming Window

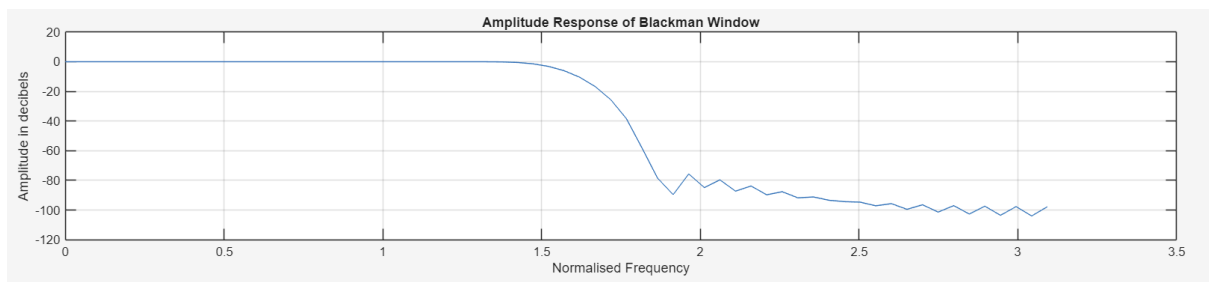


Figure 13: Amplitude Response for Blackmann Window

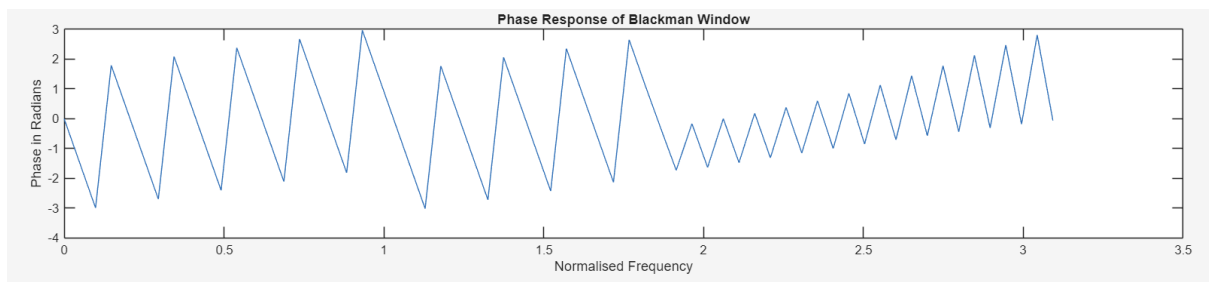


Figure 14: Phase Response for Blackmann Window

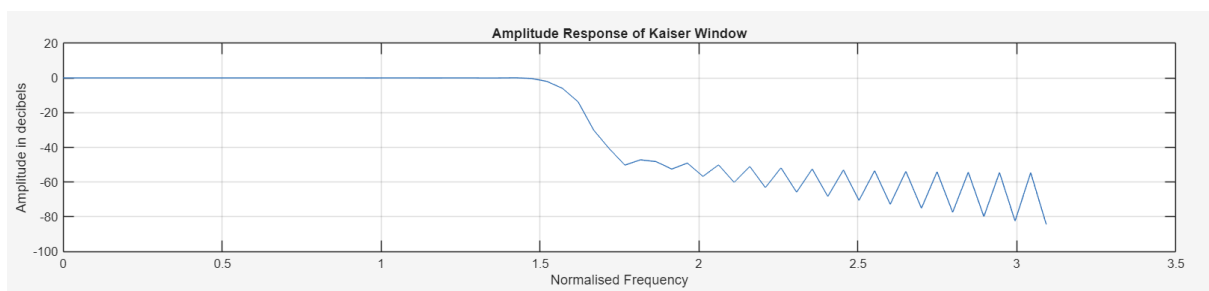


Figure 15: Amplitude Response for Kaiser Window

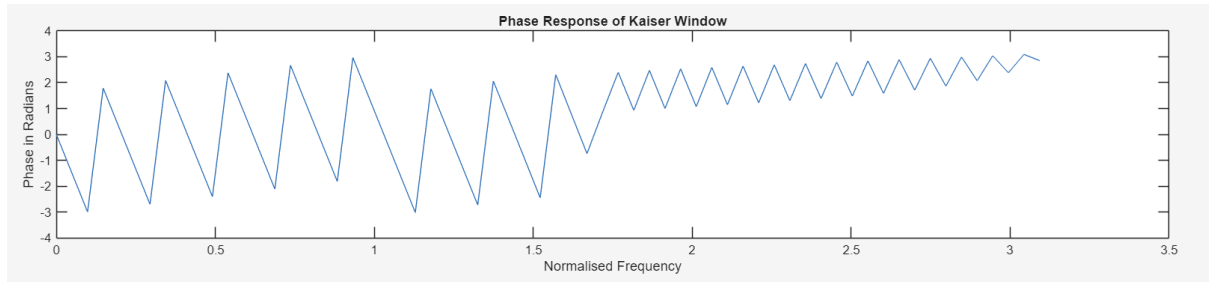


Figure 16: Phase Response for Kaiser Window

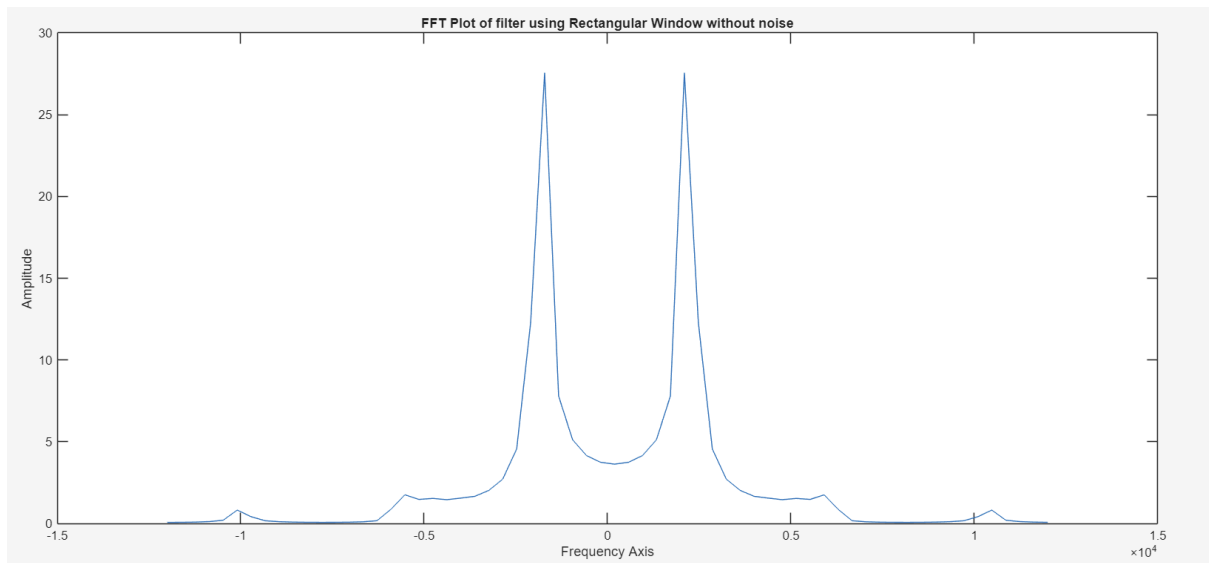


Figure 17: FFT Plot of the Output with Rectangular Window

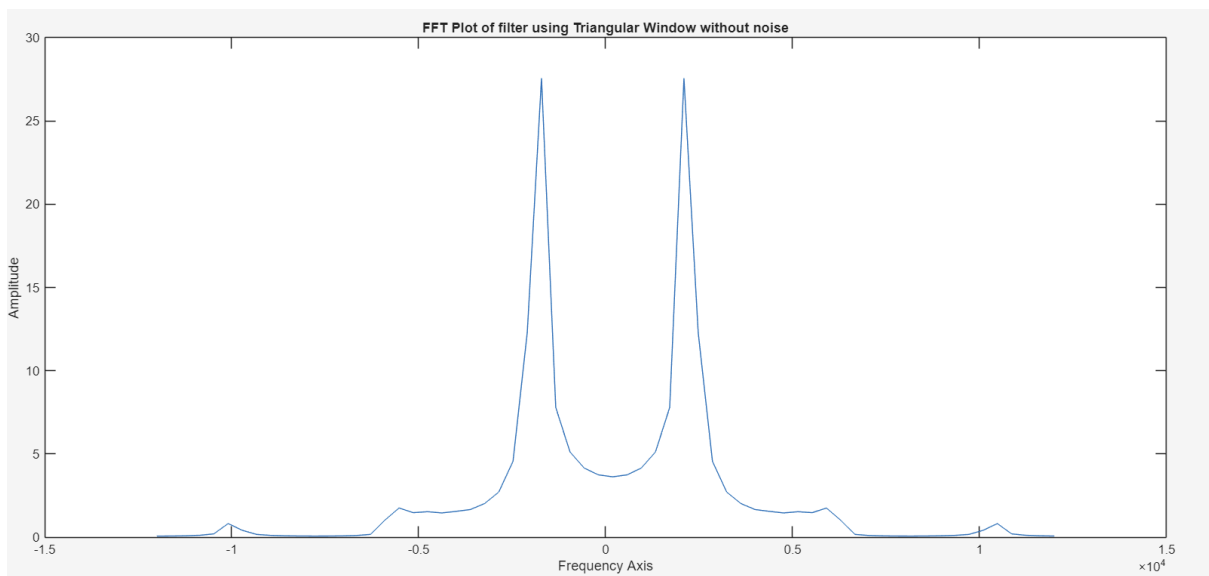


Figure 18: FFT Plot of the Output with Triangular Window

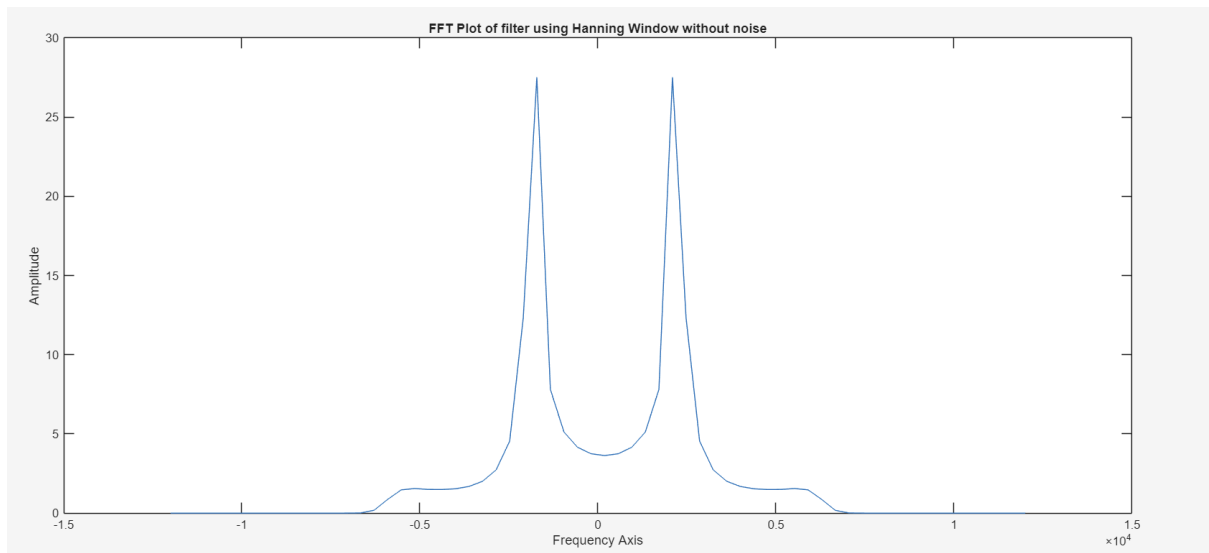


Figure 19: FFT Plot of the Output with Hanning Window

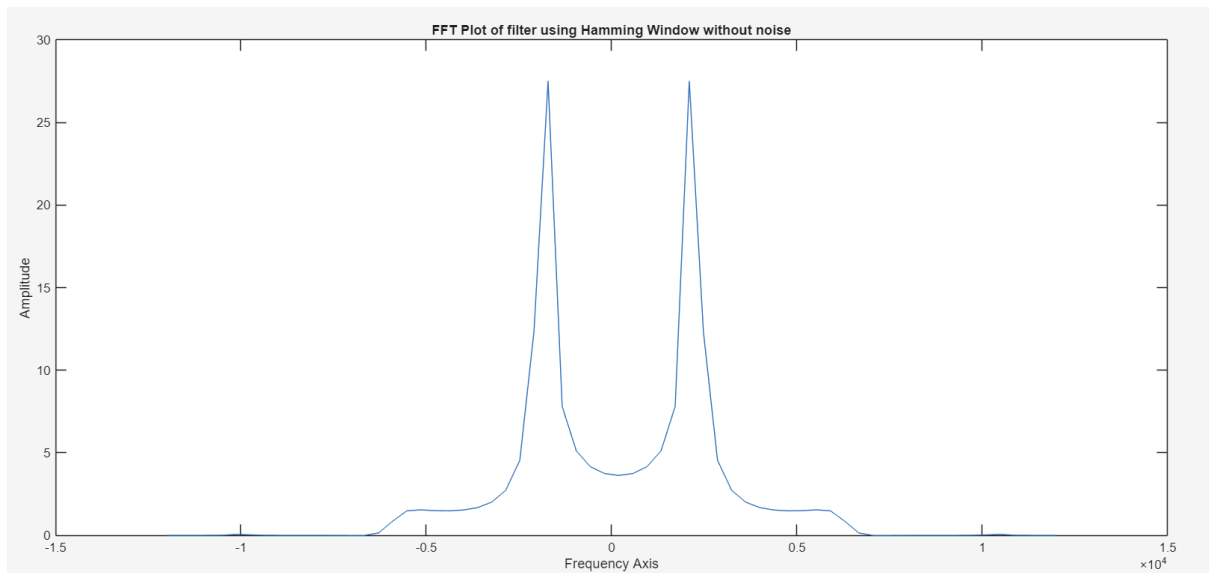


Figure 20: FFT Plot of the Output with Hamming Window

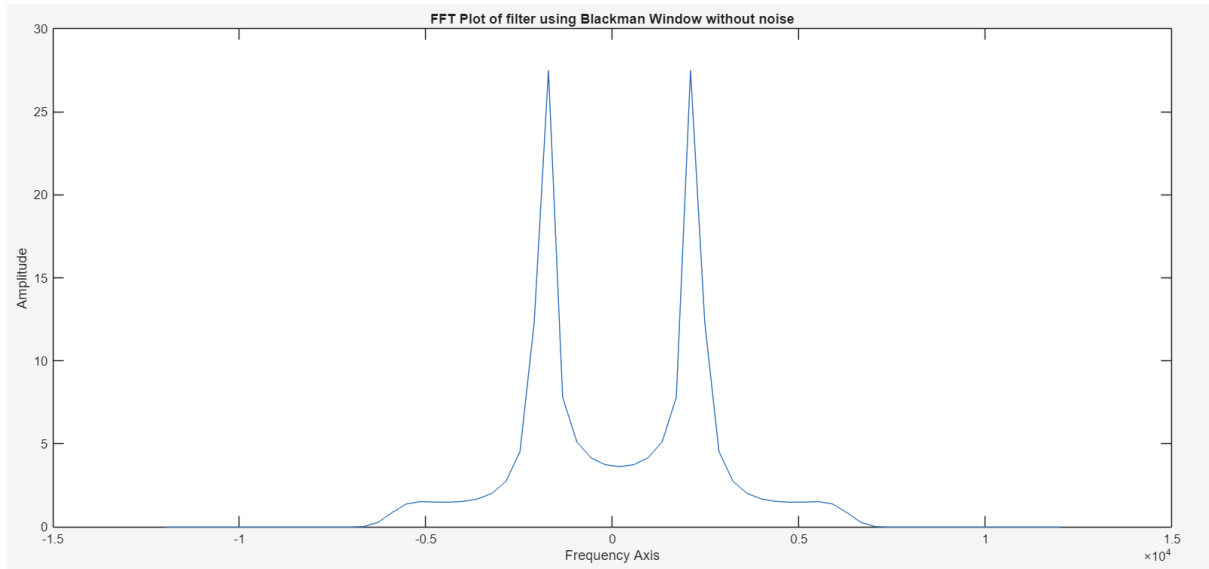


Figure 21: FFT Plot of the Output with Blackmann Window

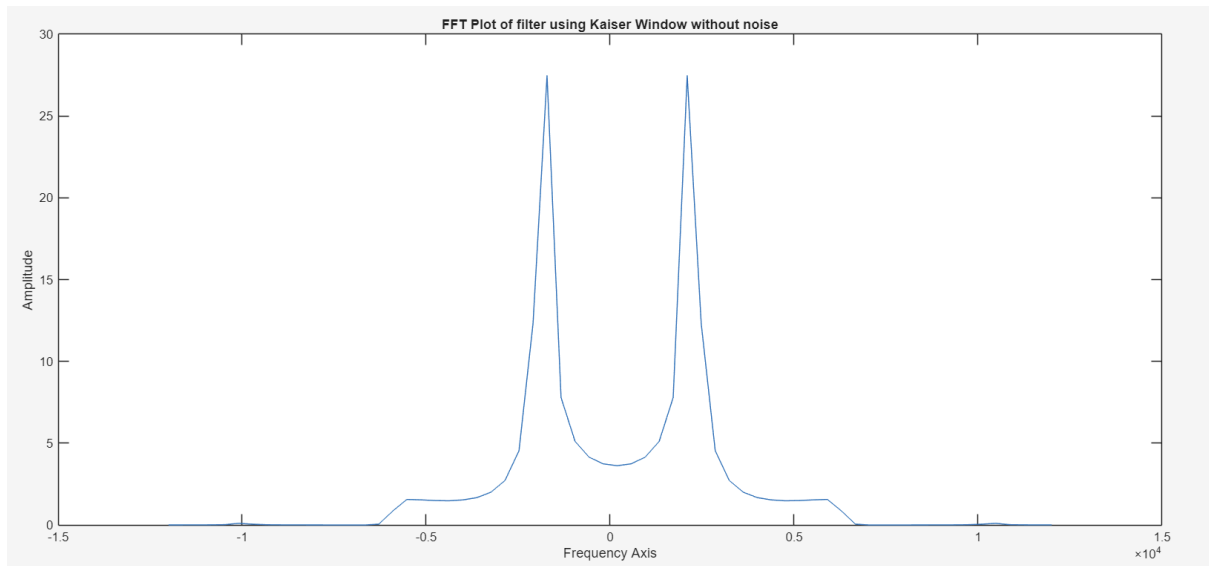


Figure 22: FFT Plot of the Output with Kaiser Window

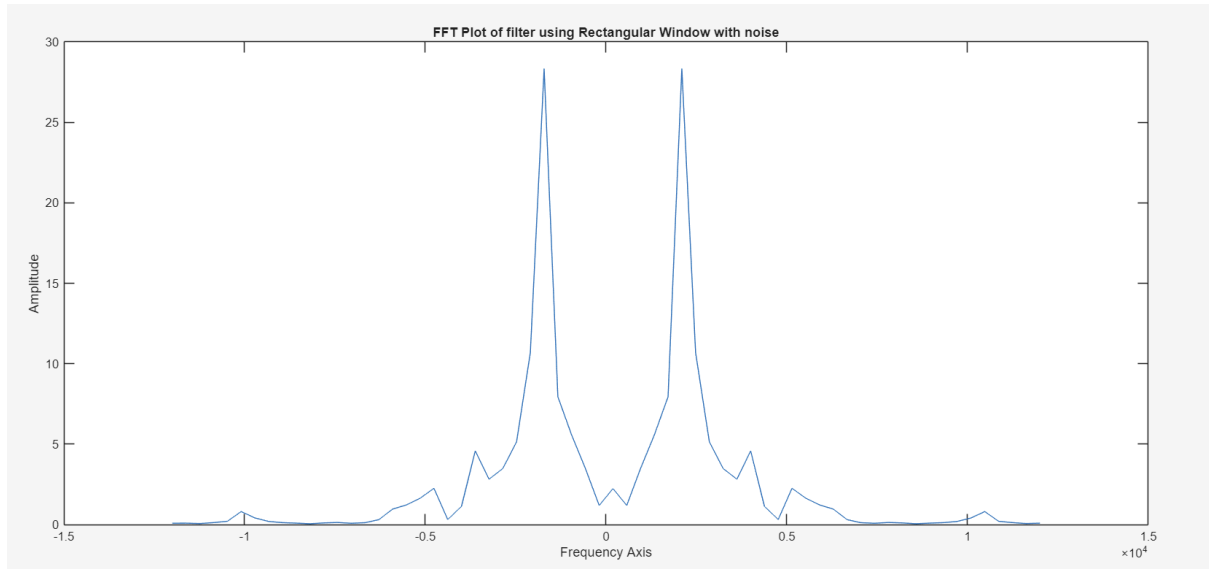


Figure 23: FFT Plot of the Output with Noise with Rectangular Window

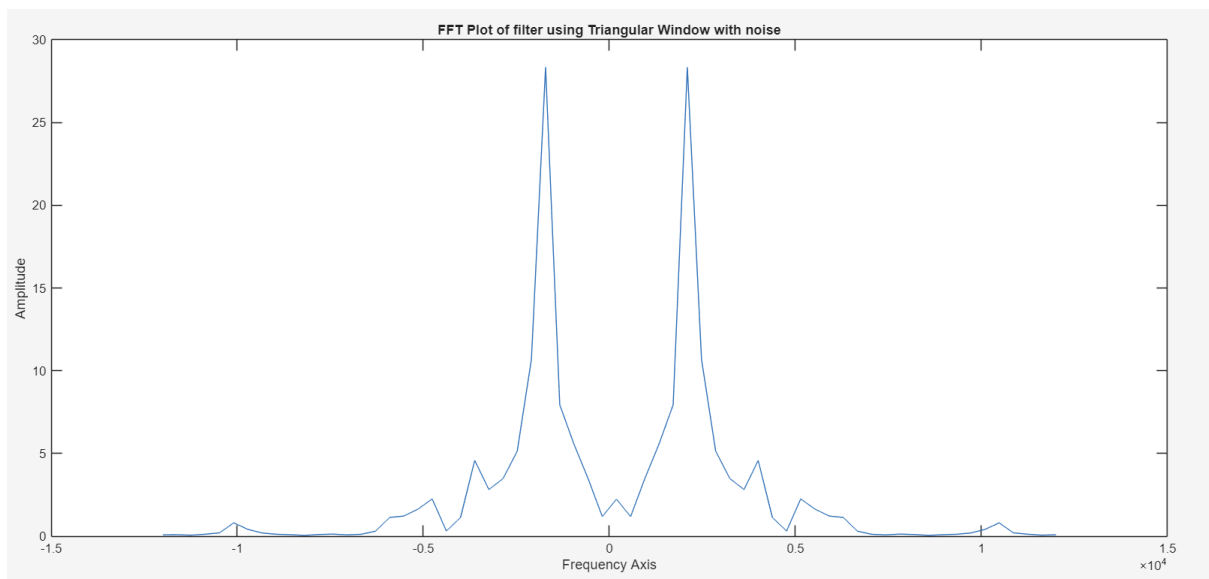


Figure 24: FFT Plot of the Output with Noise with Triangular Window

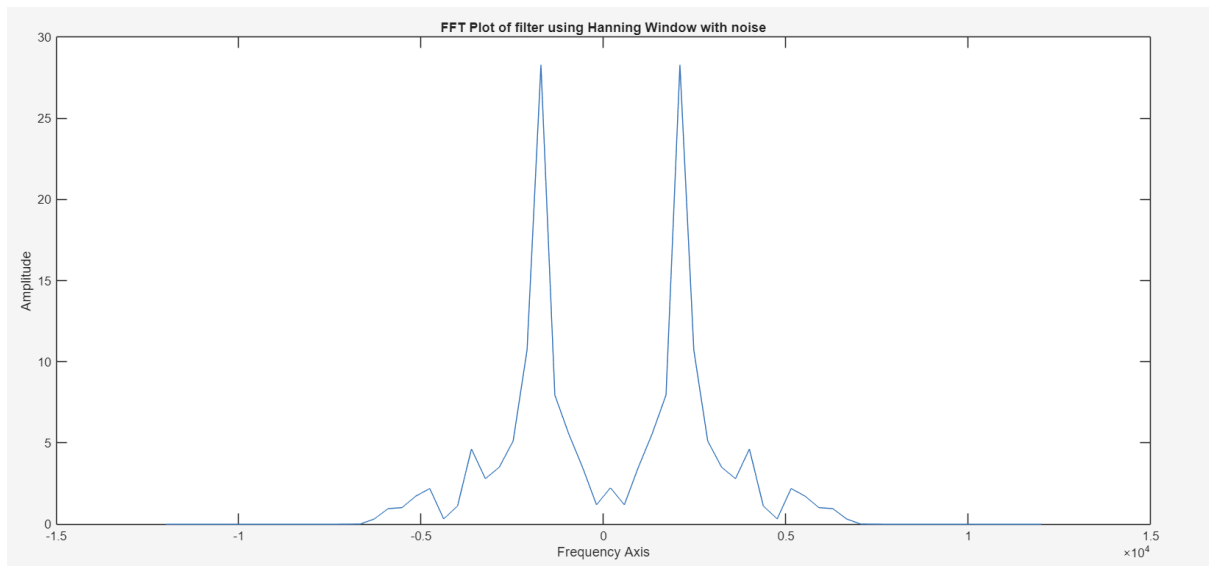


Figure 25: FFT Plot of the Output with Noise with Hanning Window

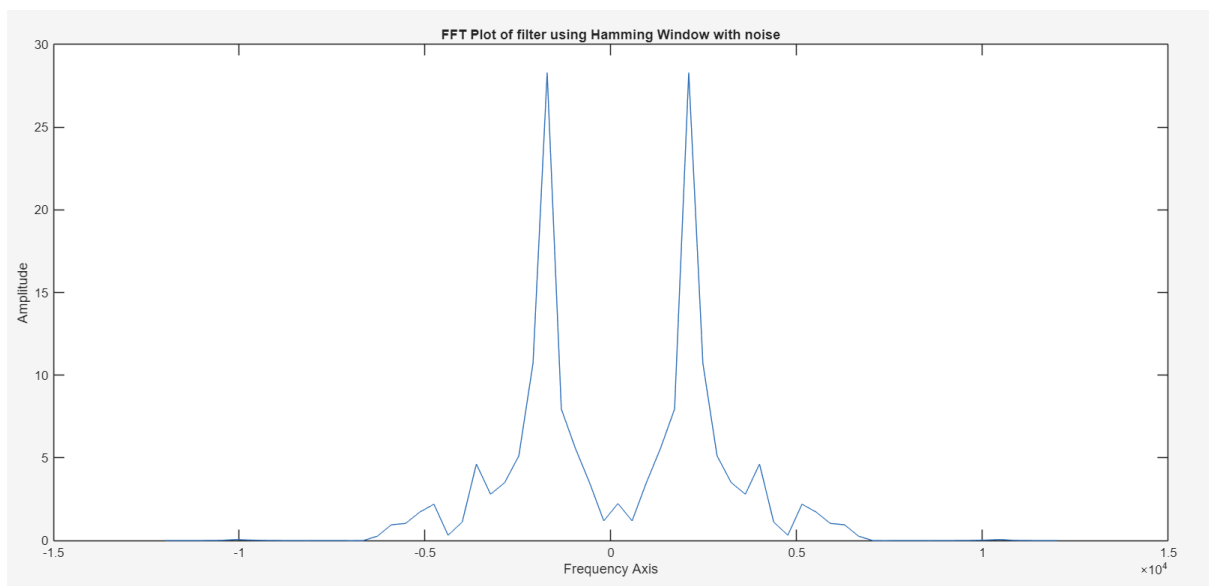


Figure 26: FFT Plot of the Output with Noise with Hamming Window

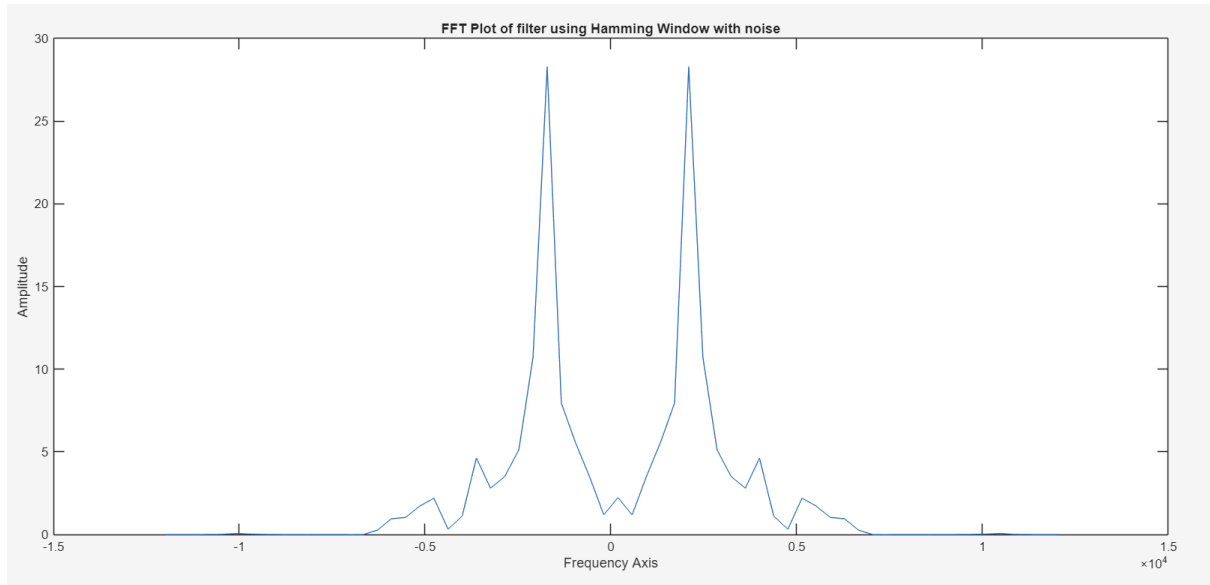


Figure 27: FFT Plot of the Output with Noise with Blackmann Window

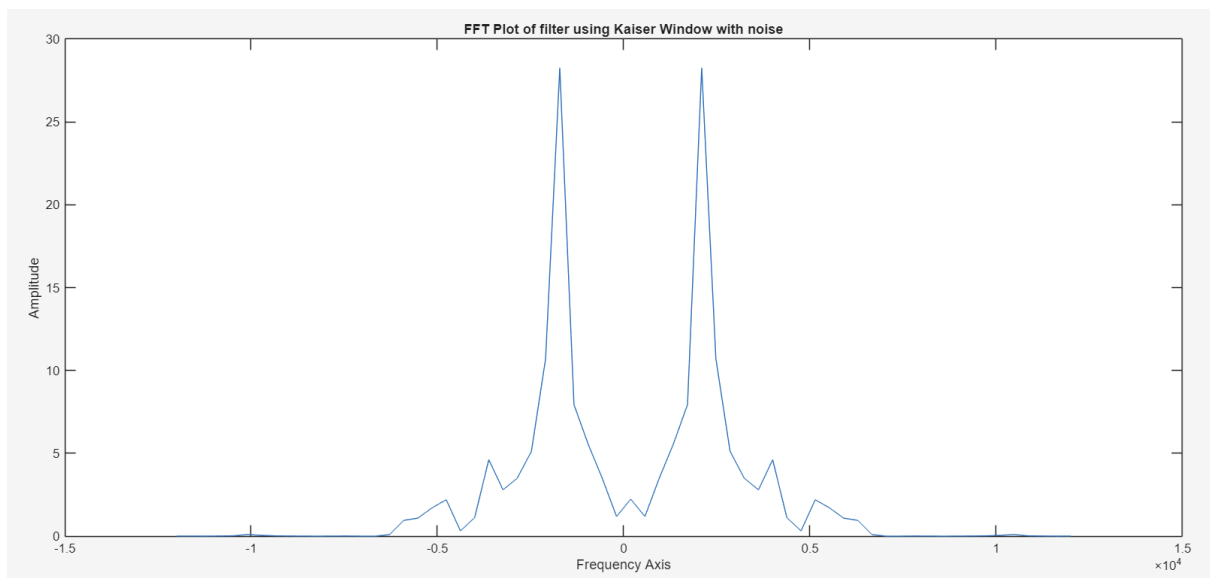


Figure 28: FFT Plot of the Output with Noise with Kaiser Window

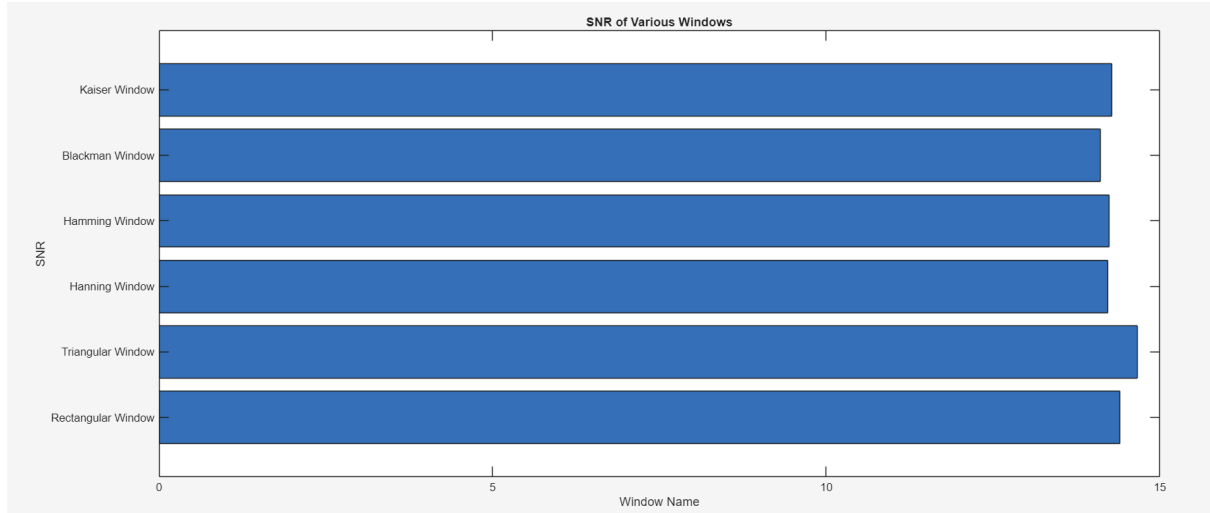


Figure 29: SNR of various windows

Table 1: Rectangular Window

N	Transition Width	Peak First Lobe	Maximum Stop Band Attenuation
8	1.3498	-39.1854	-71.718
64	0.122	-47.5867	-154
512	0.007	-60.234	-237.77

Table 2: Hamming Window

N	Transition Width	Peak First Lobe	Maximum Stop Band Attenuation
8	No Minima	No Lobe	-105.617
64	0.27	-129.872	-205
512	0.005	-132	-288.289

Table 3: Hanning Window

N	Transition Width	Peak First Lobe	Maximum Stop Band Attenuation
8	No Minima	No Lobe	-149.61
64	0.2	-102	-267
512	0.04	-125	-415

Table 4: Blackman Window

N	Transition Width	Peak First Lobe	Maximum Stop Band Attenuation
8	No Minima	No Lobe	-60
64	0.27	-188	-292
512	0.05	-180	-360

Table 5: KBDWIN Window

N	Transition Width	Peak First Lobe	Maximum Stop Band Attenuation
8	No Minima	No Lobe	-55
64	0.21	-58	-303
512	0.005	-58	-392

Table 6: Kaiser Window

N	Transition Width	Peak First Lobe	Maximum Stop Band Attenuation
8	1.35	-86	-126
64	0.0798	-76.071	-165
512	0.011	-83	-261

Table 7: Rectangular Window Noise Analysis

N	Signal Power	Noise Power	SNR
8	68.6336	17.5694	5.9178
64	529.0235	160.59167	5.1775
512	4.1119e+03	1.3669e+03	4.7829

Table 8: Hamming Window Noise Analysis

N	Signal Power	Noise Power	SNR
8	21.0744	11.7046	2.5540
64	513.5115	164.8881	4.9336
512	4.1029e+03	1.3843e+03	4.7187

Table 9: Hanning Window Noise Analysis

N	Signal Power	Noise Power	SNR
8	18.9017	11.0501	2.3313
64	512.1916	154.6301	5.201
512	4.102e+03	1.3388e+03	4.8628

Table 10: Blackman Window Noise Analysis

N	Signal Power	Noise Power	SNR
8	12.1574	5.6988	3.2906
64	509.1580	178.7423	4.5463
512	4.1017e+03	1.3012e+03	4.9862

Table 11: Kaiser Window Noise Analysis

N	Signal Power	Noise Power	SNR
8	37.9624	12.6696	4.7659
64	520.0535	161.5648	5.0770
512	4.1051e+03	1.3078e+03	4.9678

Table 12: KBDWIN Window Noise Analysis

N	Signal Power	Noise Power	SNR
8	43.3110	15.7452	4.3945
64	518.0096	177.0630	4.6621
512	4.1027e+03	1.3275e+03	4.9004

Part B:

Theory

1. Parks-McClellan Algorithm Explanation

Given a closed real set $F \subset [0, \pi]$, design a filter with frequency response

$$H(\omega) = \sum_{k=0}^n h_k \cos(k\omega), \quad \omega \in F$$

to approximate the desired response $D(\omega)$ with minimal worst-case weighted error. Introduce a continuous, positive weighting function $W(\omega)$ and define the error:

$$E(\omega) = W(\omega) [H(\omega) - D(\omega)]$$

The minimax criterion is:

$$\delta = \|E(\omega)\|_{\infty, F} = \max_{\omega \in F} |E(\omega)|$$

The unique minimax solution is guaranteed by the Alternation Theorem, which states that there exist $n + 2$ frequencies $\omega_0 < \omega_1 < \dots < \omega_{n+1}$ in F such that

$$E(\omega_i) = -E(\omega_{i+1}) = \pm\delta, \quad i = 0, 1, \dots, n$$

The optimal coefficients $h_0, \dots, h_n$ and error δ satisfy:

$$\begin{bmatrix} 1 & \cos(\omega_0) & \cdots & \cos(n\omega_0) & \frac{(-1)^0}{W(\omega_0)} \\ 1 & \cos(\omega_1) & \cdots & \cos(n\omega_1) & \frac{(-1)^1}{W(\omega_1)} \\ \vdots & \vdots & & \vdots & \vdots \\ 1 & \cos(\omega_{n+1}) & \cdots & \cos(n\omega_{n+1}) & \frac{(-1)^{n+1}}{W(\omega_{n+1})} \end{bmatrix} \begin{bmatrix} h_0 \\ h_1 \\ \vdots \\ h_n \\ \delta \end{bmatrix} = \begin{bmatrix} D(\omega_0) \\ D(\omega_1) \\ \vdots \\ D(\omega_{n+1}) \end{bmatrix}$$

For better numerical stability, coefficients can be recovered using barycentric Lagrange interpolation. Let $x_k = \cos(\omega_k)$ and define barycentric weights:

$$w_k = \prod_{\substack{i=0 \\ i \neq k}}^{n+1} (x_k - x_i)^{-1}$$

The filter response is then interpolated as:

$$H(\omega) = \frac{\sum_{k=0}^{n+1} w_k \frac{c_k}{x - x_k}}{\sum_{k=0}^{n+1} \frac{w_k}{x - x_k}}$$

where $x = \cos(\omega)$ and

$$c_k = D(\omega_k) - \frac{(-1)^k \delta}{W(\omega_k)}$$

After each iteration, find the extremal points (local maxima/minima) of $|E(\omega)|$ to update the set $\{\omega_0, \dots, \omega_{n+1}\}$, using root-finding or interval subdivision strategies.

Iterate the above process:

$$H^{(k)}(\omega) \rightarrow H^*(\omega)$$

until the alternation and minimax property are satisfied:

$$\max_{\omega \in F} |W(\omega) (H^*(\omega) - D(\omega))| \leq A\theta^k, \quad \text{for } A > 0, 0 < \theta < 1$$

For high-degree filters, barycentric interpolation and parallelized root-finding substantially improve numerical robustness and efficiency.

1. Choose $n + 2$ initial extremal frequencies ω_k (often uniform distribution, but reference scaling is better for convergence).
2. Solve the minimax system for h_k and δ .
3. Use barycentric interpolation to evaluate $H(\omega)$.
4. Locate new extremal points and update ω_k .
5. Repeat until convergence.
6. Recover final coefficients using the Inverse Discrete Fourier Transform or Clenshaw's algorithm for Chebyshev polynomials.

Least Squares Algorithm in Digital Signal Processing

In DSP, the Least Squares (LS) algorithm is a fundamental approach used extensively for designing finite impulse response (FIR) filters and system identification. The goal is to find filter coefficients that minimize the total squared error between the desired frequency response and the actual filter response across a specified frequency range.

Problem Setup Consider designing an FIR filter of order M with symmetric impulse response coefficients $h(n), n = 0, 1, \dots, M$ to approximate a desired frequency response $D(\omega)$ over the normalized frequency band $\omega \in [0, \pi]$.

The actual amplitude response of the Type I linear-phase FIR filter can be expressed as

$$A(\omega) = \sum_{n=0}^M a(n) \cos(n\omega)$$

where the modified coefficients $a(n)$ relate to $h(n)$ as

$$a(0) = h(M/2), \quad a(n) = 2h(M/2 - n), \quad n = 1, 2, \dots, M/2$$

assuming M is even for simplicity.

Weighted Least Squares Error Criterion The weighted integral squared error is defined as

$$E^2 = \int_0^\pi W(\omega) [A(\omega) - D(\omega)]^2 d\omega$$

where $W(\omega) \geq 0$ is a weighting function to emphasize the importance of errors in different frequency regions, allowing sharper control over passband and stopband behavior.

Matrix Formulation Discretizing the frequency band into N sample points $\{\omega_k\}_{k=0}^{N-1}$, the error expression becomes

$$E^2 \approx \sum_{k=0}^{N-1} W(\omega_k) \left[\sum_{n=0}^M a(n) \cos(n\omega_k) - D(\omega_k) \right]^2$$

Let

$$\mathbf{a} = [a(0), a(1), \dots, a(M)]^T, \quad \mathbf{d} = [D(\omega_0), D(\omega_1), \dots, D(\omega_{N-1})]^T$$

and define the matrix

$$\mathbf{C}_{k,n} = \sqrt{W(\omega_k)} \cos(n\omega_k)$$

for $k = 0, \dots, N-1$ and $n = 0, \dots, M$.

Then the error can be written as

$$E^2 = \|\mathbf{C}\mathbf{a} - \mathbf{d}_w\|_2^2$$

where $\mathbf{d}_w = [\sqrt{W(\omega_0)}D(\omega_0), \dots, \sqrt{W(\omega_{N-1})}D(\omega_{N-1})]^T$.

Optimal Filter Coefficients The least squares estimate minimizing E^2 is given by solving the normal equations

$$\mathbf{C}^T \mathbf{C} \mathbf{a} = \mathbf{C}^T \mathbf{d}_w$$

which yields

$$\mathbf{a} = (\mathbf{C}^T \mathbf{C})^{-1} \mathbf{C}^T \mathbf{d}_w$$

Interpretation This solution ensures that the filter approximates the desired frequency response in the mean-square sense, balancing errors over the entire frequency band weighted by $W(\omega)$.

Constrained Least Squares Extension Sometimes, constraints such as nulls at specific frequencies or maximum ripple bounds are included, resulting in constrained least-squares problems. These are solved by incorporating linear constraints on $\mathbf{a}$ or utilizing modified quadratic programming techniques.

Practical Note Due to symmetry and the cosine basis, $\mathbf{C}^T \mathbf{C}$ is often Toeplitz and well-conditioned, allowing efficient numerical solution methods.

Summary Least squares FIR filter design provides:

- Flexibility to approximate arbitrary desired frequency responses.
- Explicit control through weight function $W(\omega)$ of passband and stopband error emphasis.
- A linear algebra formulation suitable for implementation in MATLAB and other DSP tools.

The algorithm is integral to many DSP filter design toolboxes such as MATLAB's `firls` function and forms a foundation upon which other methods like Parks–McClellan build.

Code

```

clear;
close all;
N = 32;
wp1 = 0.3;
wp2 = 0.5;
wp3 = 0.65;
wp4 = 0.75;
ws1 = 0.25;
ws2 = 0.55;
ws3 = 0.62;
ws4 = 0.8;
w_c = 0.5;
w_p = 0.6;
figure(1);
least_squares_filter = firls(N,[0,ws1,wp1,wp2,ws2,ws3,wp3,wp4,ws4,1],[0.1,0.1,0.6,0.6,0.017,0.017,1,1,0.056,0.056]);
freqz(least_squares_filter,1);
title("Using Least Square Filter");
figure(2);
pm_filter = firpm(N,[0,ws1,wp1,wp2,ws2,ws3,wp3,wp4,ws4,1],[0.1,0.1,0.6,0.6,0.017,0.017,1,1,0.056,0.056]);
freqz(pm_filter,1);
title("Using PM Algorithm");
figure(3);
least_square_lowpass_filter = firls(N,[0,w_c,w_p,1],[1,1,0.005,0.005]);
freqz(least_square_lowpass_filter,1);
title("Using Least Square Filter on Low Pass Filter");
figure(4);
pm_low_pass_filter = firpm(N,[0,w_c,w_p,1],[1,1,0.005,0.005]);
freqz(pm_low_pass_filter,1);
title("Using PM Algorithm on Low Pass Filter");

```

Figure 30: Code for PM Algorithm and Least Square Algorithm for Custom Filter and Low Pass Filter

```

N = 32;
wp1 = 0.3;
wp2 = 0.5;
wp3 = 0.65;
wp4 = 0.75;
ws1 = 0.25;
ws2 = 0.55;
ws3 = 0.62;
ws4 = 0.8;
magnitude = [0.1,0.6,0.017,1,0.056];
A = [0.25*pi, 0.1 *pi, 0.035*pi,0.05*pi, 0.1*pi];
B = [0,0.4*pi,0.585*pi,0.7*pi,0.9*pi];
function output = compute(n,A,B,magnitude)
    if n==0
        n=1e-7;
    end
    output = 2*magnitude*sin(n*A) * cos(B*n)/(n*pi);
end

h_n = zeros(1,N);
for i = -N/2 : N/2 - 1
    for j = 1:5
        h_n(i+N/2+1) = h_n(i+N/2+1) + compute(i,A(j),B(j),magnitude(j));
    end
end
kaiser_window = kaiser(N,2.5)';
kaiser_window_h_n = kaiser_window.*h_n;
freqz(kaiser_window_h_n,1);
title("Kaiser Window");

```

Figure 31: Code for Kaiser Window on Custom Filter

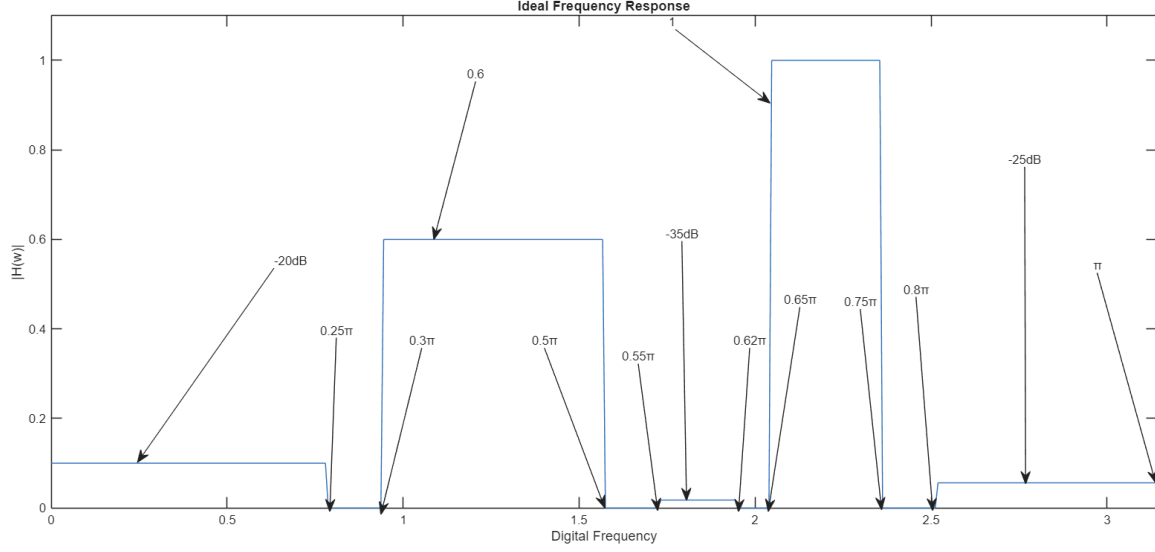


Figure 32: The Ideal Custom Filter Frequency Response to be Constructed

Inverse DTFT of an Ideal Band-Pass Filter

The frequency response of an ideal band-pass filter is defined as

$$H_{BP}(\omega) = \begin{cases} 1, & \omega_1 \leq |\omega| \leq \omega_2 \\ 0, & \text{otherwise} \end{cases}$$

where $0 \leq \omega_1 < \omega_2 \leq \pi$.

Using linearity of the DTFT, this can be expressed as the difference of two ideal low-pass filters:

$$H_{BP}(\omega) = H_{LP,\omega_2}(\omega) - H_{LP,\omega_1}(\omega)$$

The inverse discrete-time Fourier transform (IDTFT) of the ideal low-pass filter with cutoff ω_c is

$$h_{LP,\omega_c}[n] = \frac{\sin(\omega_c n)}{\pi n}, \quad n \neq 0$$

and

$$h_{LP,\omega_c}[0] = \frac{\omega_c}{\pi}$$

Therefore, the impulse response of the ideal band-pass filter is expressed as

$$h_{BP}[n] = \frac{2}{\pi n} \cos\left(\frac{\omega_2 + \omega_1}{2}n\right) \sin\left(\frac{\omega_2 - \omega_1}{2}n\right), \quad n \neq 0$$

and

$$h_{BP}[0] = \frac{\omega_2 - \omega_1}{\pi}$$

Now since we have 5 such Band Pass Filters, taking A_m to be the Amplitude of each BandPass Filter, center frequency $\omega_{cm} = \frac{\omega_2 + \omega_1}{2}$ and Bandwidth $B_m = \frac{\omega_2 - \omega_1}{2}$

Consider the impulse response of a sum of $M = 5$ ideal band-pass filters with amplitudes A_m , center frequencies $\omega_{c,m}$, and bandwidths B_m , where $m = 1, 2, \dots, 5$.

Each individual band-pass filter impulse response is:

$$h_m[n] = \frac{2A_m}{\pi n} \cos(\omega_{c,m}n) \sin\left(\frac{B_m}{2}n\right), \quad n \neq 0$$

and at $n = 0$:

$$h_m[0] = \frac{A_m B_m}{\pi}$$

The overall impulse response is the sum:

$$h[n] = \sum_{m=1}^5 h_m[n] = \sum_{m=1}^5 \begin{cases} \frac{2A_m}{\pi n} \cos(\omega_{c,m}n) \sin\left(\frac{B_m}{2}n\right), & n \neq 0 \\ \frac{A_m B_m}{\pi}, & n = 0 \end{cases}$$

We used this method to construct the coefficients for the Custom Filter

Plots

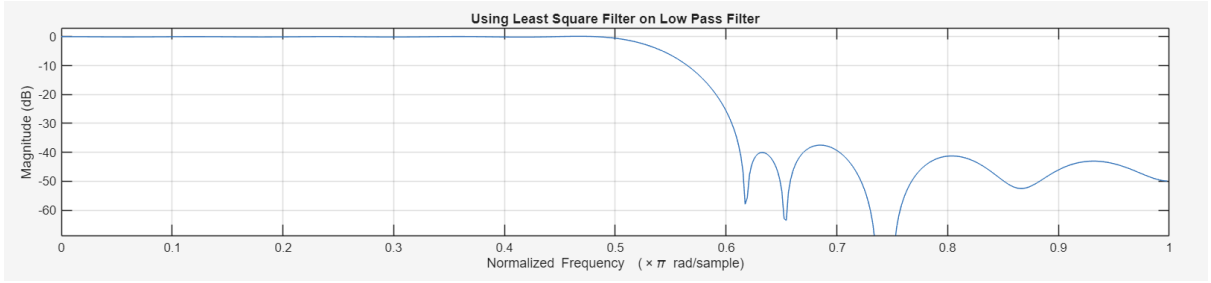


Figure 33: Amplitude Response of the Low Pass Filter Designed using Least Squares Algorithm

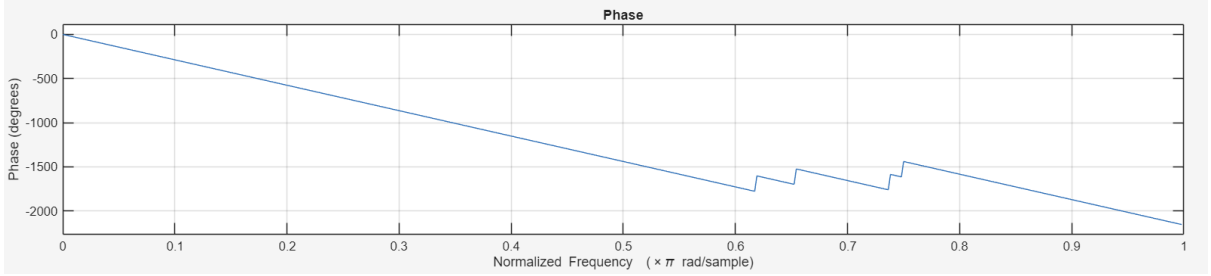


Figure 34: Phase Response of the Low Pass Filter Designed using Least Squares Algorithm

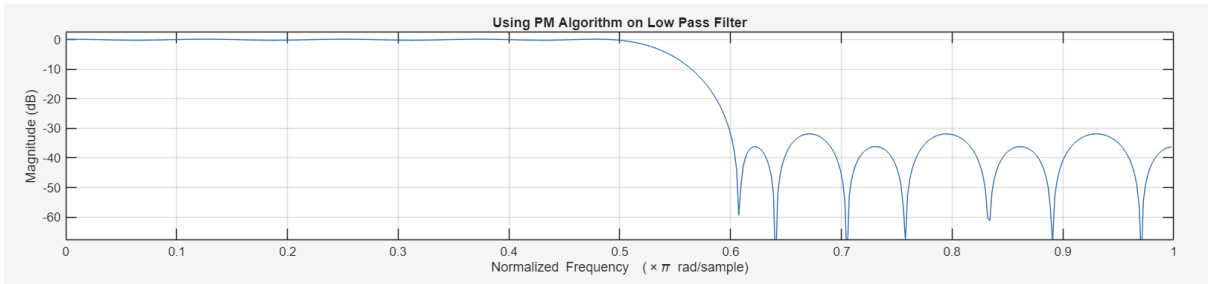


Figure 35: Amplitude Response of the Low Pass Filter Designed using Parks-McClellan filter design algorithm

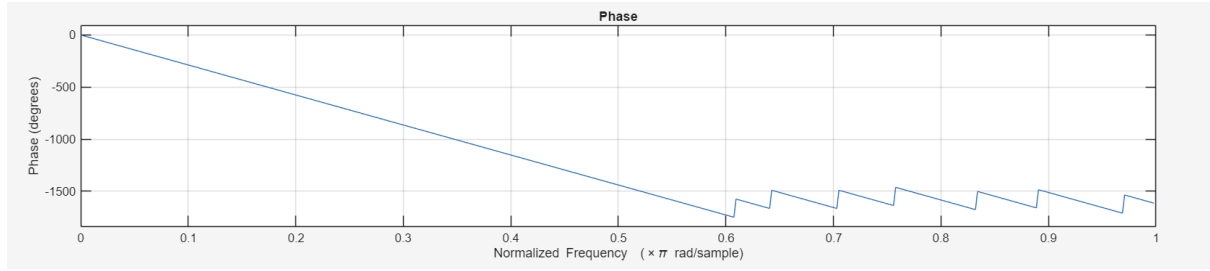


Figure 36: Phase Response of the Low Pass Filter Designed using Parks–McClellan filter design algorithm

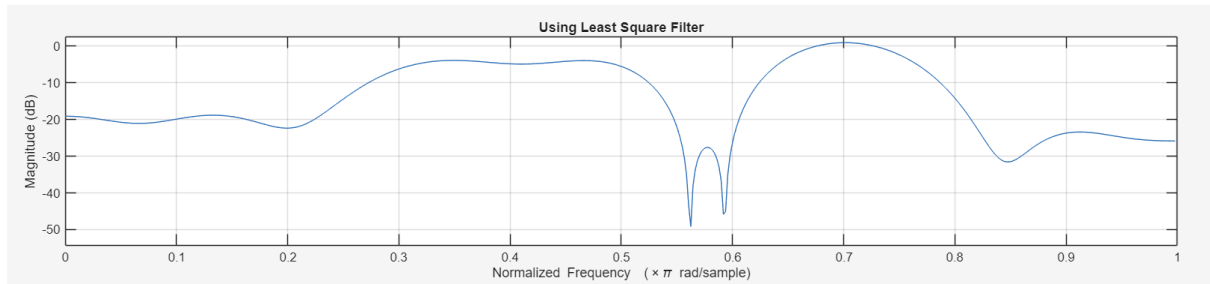


Figure 37: Amplitude Response of the Custom Filter Designed using Least Squares Algorithm

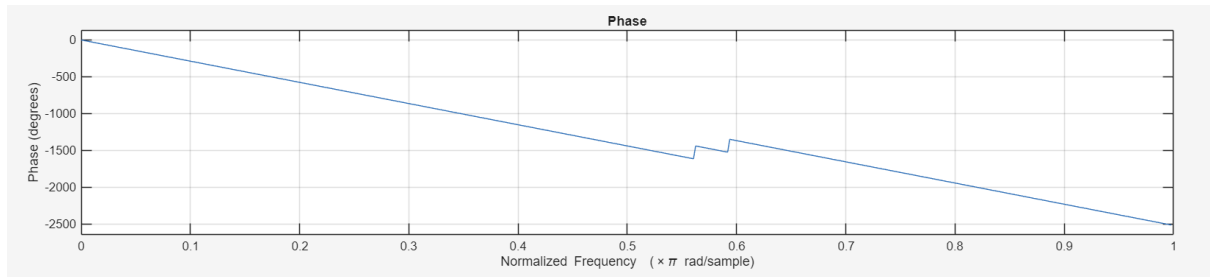


Figure 38: Phase Response of the Custom Filter Designed using Least Squares Algorithm

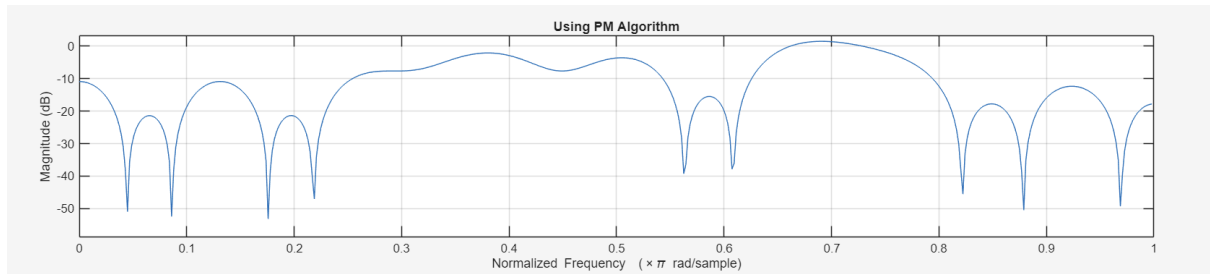


Figure 39: Amplitude Response of the Custom Filter Designed using Parks–McClellan filter design algorithm

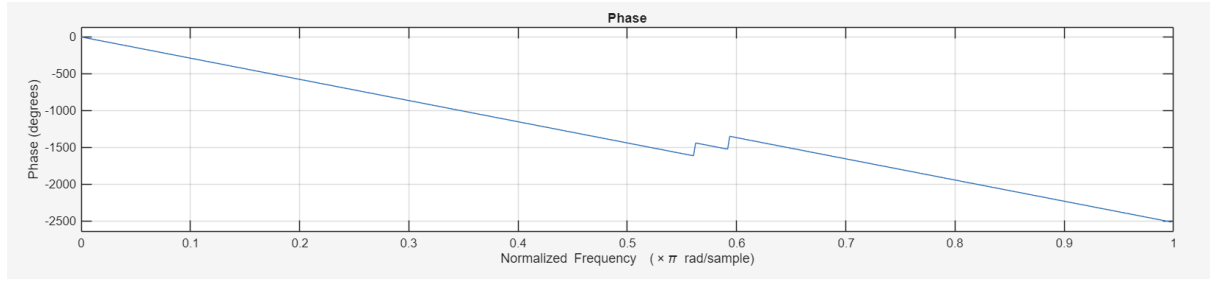


Figure 40: Phase Response of the Custom Filter Designed using Parks–McClellan filter design algorithm

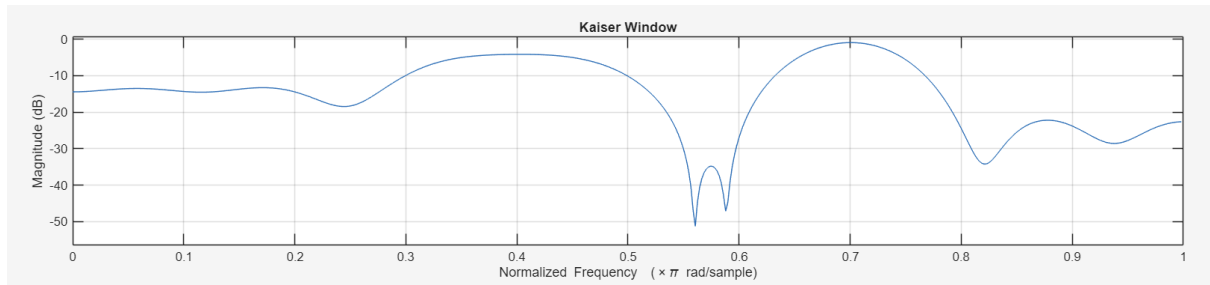


Figure 41: Amplitude Response of the Custom Filter Designed using Kaiser Window

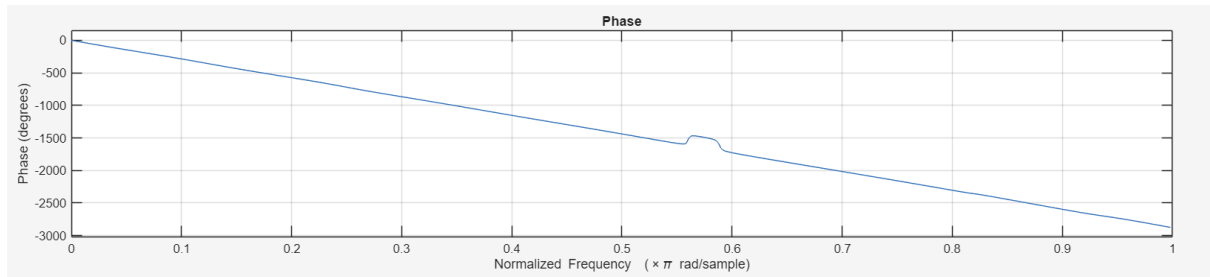


Figure 42: Phase Response of the Custom Filter Designed using Kaiser Window

Part C:

Theory

Comprehensive Overview of Finite-Precision Numerical Effects in Digital Signal Processing

Digital Signal Processing (DSP) theory is predominantly developed in the continuous domain of complex mathematics, assuming infinite precision for both signal values and system parameters. However, the physical implementation of these systems—whether in dedicated hardware (ASICs, FPGAs) or software on general-purpose processors—is confined to the discrete world of binary digits (bits) of finite length. This fundamental limitation gives rise to a suite of non-ideal behaviors known as **Finite Word Length Effects**. Understanding these effects is not merely an academic exercise but a critical pillar of practical engineering, separating a theoretical design from a robust, functional

product. The consequences of ignoring these effects range from slight performance degradation to complete system failure (instability).

Number Representations: The Foundation of Digital Arithmetic

The choice of number representation dictates every aspect of a system’s numerical behavior. It is the first and most critical decision in a fixed-point design.

Fixed-Point Representation

This representation is analogous to using a ruler with fixed, evenly spaced markings. The position of the “binary point” is implicit and fixed by the designer.

Mathematical Formulation: A fixed-point number system is defined by four parameters: the word length (number of bits, B), the location of the binary point, and whether the format is signed or unsigned. A common signed format is the two’s complement fractional format, where the most significant bit (MSB) is the sign bit, and the value is interpreted as:

$$X = -b_0 + b_1 2^{-1} + b_2 2^{-2} + \cdots + b_{B-1} 2^{-(B-1)}$$

Here, the binary point is immediately to the right of the sign bit b_0 . This is often called the $Q1.(B-1)$ format, indicating 1 integer bit (including sign) and $(B-1)$ fractional bits. The representable range is:

$$[-1, 1 - 2^{-(B-1)}]$$

Quantization Step (Δ): The fundamental precision of the system is the Least Significant Bit (LSB) value, which is $\Delta = 2^{-(B-1)}$ for the $Q1.(B-1)$ format. This is a constant. The quantization error, $e = Q(x) - x$, when using rounding is uniformly distributed in $[-\Delta/2, \Delta/2]$. When using truncation, the error for two’s complement is also in $[-\Delta, 0]$. This constant step size is known as uniform quantization.

Dynamic Range and Scaling: The limited, fixed range is the primary challenge. The designer must perform dynamic range analysis to determine appropriate scaling factors for every signal in the system. The goal is to maximize signal utilization (to improve the Signal-to-Quantization-Noise Ratio, SQNR) while guaranteeing that no operation results in overflow. Overflow is catastrophic in fixed-point arithmetic as it wraps values from large positive to large negative (or vice-versa), causing severe nonlinear distortion. Strategies include incorporating saturation arithmetic or using block-floating-point techniques.

Applications: Ubiquitous in high-throughput, low-power, and cost-sensitive applications like audio codecs, automotive systems, and embedded control due to its hardware efficiency.

Floating-Point Representation

This representation is analogous to scientific notation, allowing the “binary point” to float, thereby achieving a massive dynamic range.

IEEE 754 Standard Format: A number X is represented as:

$$X = (-1)^s \cdot M \cdot 2^{(E-\text{Bias})}$$

- **Sign Bit (s):** 1 bit.
- **Exponent (E):** 8 bits for single-precision (32-bit total), 11 bits for double-precision (64-bit total). The bias (127 for single-precision) is subtracted to allow for both positive and negative exponents.
- **Significand/Mantissa (M):** 23 bits for single-precision, 52 for double-precision. It represents a fractional value between 1.0 and 2.0, with an implied leading ‘1’ (the “normalized” form).

Non-Uniform Quantization: The quantization step Δ is not constant; it is proportional to the magnitude of the number itself:

$$\Delta(X) \approx |X| \cdot 2^{-\text{MantissaBits}}$$

This means the relative error is nearly constant, but the absolute error grows with the number. This is a non-uniform quantization.

Advantages and Overhead: The primary advantage is the dramatic simplification of the numerical design process. The risk of overflow and underflow is immensely reduced. The trade-off is significantly higher hardware complexity: floating-point multipliers and adders are larger, consume more power, and have longer latencies than their fixed-point counterparts.

Quantization in System Implementation: The Sources of Error

Quantization error injects noise into the system at two fundamental points:

Coefficient Quantization (Deterministic Error): The filter coefficients a_k, b_k calculated by design software (e.g., using a Chebyshev or elliptic method in MATLAB) are real numbers. They must be quantized to the finite word length of the target hardware. This process alters the transfer function $H(z)$ from its ideal form. The error is deterministic; once the coefficients are set, the deviation in the frequency response is fixed. This is a problem of accuracy.

Signal Quantization (Stochastic Error): This occurs during signal processing.

- **A/D Conversion:** The initial conversion from analog to digital introduces an error that can often be modeled as additive white noise.
- **Arithmetic Operations:** After a multiplication, the product has a word length of $2B$ bits. This result must be quantized back to B bits to be stored in a register or memory. This process, whether by rounding or truncation, introduces a small error at each operation. Since these operations occur on data sequences that can be considered random, the accumulated error is typically modeled as a random process—specifically, a white noise source with zero mean and a variance of $\Delta^2/12$ for rounding.

This is a problem of precision and noise accumulation.

The Effects of Coefficient Quantization

This section addresses how the deterministic error of quantizing coefficients impacts the system’s intended behavior.

Effects in IIR Systems: The Peril of Pole Migration

IIR filters are characterized by their feedback paths, which create poles in the transfer function. The stability and frequency response of an IIR filter are exquisitely sensitive to the precise location of these poles within the z -plane.

Sensitivity Analysis: Consider a direct-form IIR filter with denominator polynomial:

$$D(z) = 1 + a_1 z^{-1} + \cdots + a_N z^{-N}$$

The roots of $D(z) = 0$ are the poles. The sensitivity of a particular pole p_i to a small change in a coefficient a_k is given by a partial derivative $\frac{\partial p_i}{\partial a_k}$. For high-order filters (N large) with tightly clustered poles (required for sharp transition bands), this sensitivity can be extremely high. A change in the coefficient a_k on the order of the quantization step Δ can lead to a displacement of the pole p_i that is magnified by a factor related to the reciprocal of the distance between poles. This can pull a pole originally just inside the unit circle (stable) to outside the unit circle (unstable).

Direct-Form Catastrophe: Implementing a high-order IIR filter in direct form I or II is therefore highly discouraged. The coefficient quantization alone can render the filter unusable, even before considering signal quantization effects.

The Elliptic Filter Example and the Second-Order Section (SOS) Solution

The Problem Exemplified: An elliptic filter achieves the sharpest possible transition band for a given filter order. This optimality comes at a cost: the poles of an elliptic filter are distributed very close to the unit circle and are highly sensitive. Quantizing the coefficients of an 8th-order elliptic direct-form filter to 16 bits can cause the frequency response to deviate wildly from the specification, with passband ripple exploding and stopband attenuation collapsing.

The Cascade of Second-Order Sections (SOS): The robust solution is to avoid implementing the high-order polynomial directly. Instead, the transfer function $H(z)$ is factored into a product of first-order and second-order transfer functions:

$$H(z) = G \cdot H_1(z) \cdot H_2(z) \cdots H_{N_{sec}}(z)$$

Each second-order section (biquad) has the form:

$$H_i(z) = \frac{b_{i0} + b_{i1}z^{-1} + b_{i2}z^{-2}}{1 + a_{i1}z^{-1} + a_{i2}z^{-2}}$$

Why is this superior? The sensitivity of the poles of the overall filter is now distributed. The poles of the overall system are the roots of the product of the second-order denominator polynomials. Crucially, the poles implemented by a single SOS depend only on its two denominator coefficients a_{i1} and a_{i2} . Quantizing these coefficients will affect only that specific pair of poles. The error is localized.

Pole Pairing and Section Ordering: When decomposing into a cascade:

- **Pole-Zero Pairing:** Pair the complex-conjugate pole pair closest to the unit circle with the closest complex-conjugate zero pair. This minimizes the peak gain within each section.
- **Section Ordering:** Order sections in increasing peak gain, placing the highest-gain section last to prevent it from amplifying the noise of all others.

Effects in FIR Systems: The Stability Advantage

FIR filters, having no feedback (all poles at $z = 0$), are unconditionally stable. Coefficient quantization cannot induce instability.

Impact on Frequency Response: The ideal impulse response $h[n]$ is infinite-precision. The quantized impulse response is:

$$\hat{h}[n] = h[n] + e_c[n]$$

where $e_c[n]$ is the coefficient quantization error. The resulting frequency response is:

$$\hat{H}(\omega) = H(\omega) + E_c(\omega)$$

where $E_c(\omega)$ is the Fourier transform of the error sequence.

The Optimum FIR Filter and Preservation of Linear Phase

Degradation of Optimality: The Parks-McClellan (Remez exchange) algorithm produces an equiripple FIR filter that is optimal in the minimax sense. Quantizing the coefficients disrupts this balance, causing the equiripple property to be lost. The resulting filter will have worse passband ripple/stopband attenuation.

Preservation of Linear Phase: Linear phase is achieved by designing the impulse response to be symmetric ($h[n] = h[N - 1 - n]$). If quantization respects this symmetry, then the quantized response $\hat{h}[n]$ is also symmetric, ensuring exact linear phase even though magnitude response degrades.

Elaborate Mitigation Strategies for Finite Word Length Effects

Finite word length effects can significantly degrade the performance of digital filters and DSP systems. Various strategies have been developed to mitigate these effects, each with its own advantages, limitations, and applications.

- **Increasing Word Length:**

The simplest and most direct method to reduce quantization error is to use more bits in the representation of both coefficients and signals. For example, moving from 16-bit to 32-bit representation increases the number of quantization levels exponentially, thereby reducing the Least Significant Bit (LSB) value. This leads to a dramatic improvement in Signal-to-Quantization-Noise Ratio (SQNR) and reduces both coefficient quantization and round-off noise. However, increasing word length also increases hardware complexity, memory requirements, power consumption, and computational latency, which must be carefully balanced against the performance benefits.

- **Lattice Structures:**

Lattice structures are alternative filter implementations that are inherently robust to coefficient quantization. They are based on reflection coefficients rather than direct polynomial coefficients, which naturally limits the sensitivity of poles and zeros to quantization. An additional advantage is that lattice filters have a built-in mechanism for monitoring and ensuring stability, making them particularly suitable for high-fidelity applications such as speech processing, adaptive filters, and high-order filter designs.

- **State-Space Structures:**

State-space representations model a filter in terms of internal states and matrices (A , B , C , D), allowing a direct and flexible control over the system dynamics. By carefully choosing or orthonormalizing the state-space matrices, the propagation of round-off noise can be minimized. This approach is particularly effective for complex multi-stage systems, multi-input multi-output (MIMO) filters, and applications where precise control of internal signal energy is crucial.

- **Error Spectrum Shaping (ESS):**

Also known as noise shaping, ESS is a technique that redistributes the power spectral density of quantization noise. Instead of letting quantization noise be flat across the frequency spectrum, it is deliberately pushed out of the band of interest (e.g., outside the audio or signal band) using feedback mechanisms. This increases the effective SNR within the desired band, which is especially useful in delta-sigma modulators and high-resolution analog-to-digital converters (ADCs).

- **Dithering:**

Dithering involves adding a small, random noise signal to the input of a quantizer before quantization. This process breaks up any deterministic correlation between the signal and the quantization error, effectively converting harmful nonlinear distortion into a more benign, noise-like behavior. Dithering is widely used in audio signal processing to prevent low-level “whistling” artifacts, limit cycles, and other spurious quantization-induced phenomena. Properly designed dithering can improve perceived signal quality without significantly affecting the signal power.

Overall, these mitigation strategies are often combined depending on the application. For instance, a high-order IIR filter may be implemented using a cascade of second-order sections with lattice structures, extended word length, and dithering to achieve both numerical stability and high-fidelity performance.

Deep Dive: Quantization Effects in Parallel and Cascade IIR Structures

Cascade Structure

In a cascade realization, a higher-order IIR filter is decomposed into a series of second-order sections (SOS) or biquads. This approach is widely used because it provides better numerical stability and allows for more controlled pole-zero placements.

Coefficient Quantization Sensitivity:

One of the main benefits of the cascade structure is that coefficient quantization sensitivity is reduced. Instead of representing the filter with one set of very high-order coefficients (which would be highly sensitive to quantization), each SOS operates with lower-order coefficients that are less affected by quantization. The localization of poles within each SOS ensures that the pole radii are less perturbed by rounding or truncation, thereby improving stability.

Propagation of Noise:

However, the quantization errors introduced in one section do not remain localized. Each SOS generates round-off noise due to finite-precision arithmetic, and this noise propagates through the subsequent sections. As a result, the placement or ordering of the sections significantly influences the overall noise distribution.

- If sections with poles close to the unit circle (highly resonant sections) are placed early in the cascade, the noise from these sections is amplified by subsequent sections.
- A better strategy is to place sections with poles closer to the unit circle toward the end of the cascade, and sections with poles farther from the unit circle (less sensitive) at the beginning.
- This ordering minimizes the cumulative amplification of noise and improves the overall signal-to-noise ratio (SNR).

Thus, while coefficient sensitivity is reduced in cascade structures, careful design and ordering of sections are required to control round-off noise accumulation.

Parallel Structure

In the parallel realization, the transfer function of the IIR filter is expressed as a sum of lower-order components (often obtained via partial fraction expansion). Each branch of the parallel form is typically a second-order (or first-order) filter, and their outputs are added together at the final summing junction.

Coefficient Quantization Sensitivity:

Parallel structures also benefit from reduced sensitivity to coefficient quantization, because each branch operates with relatively simple coefficients. However, in practice, the numerator coefficients in the partial fraction decomposition may be more complex or larger in magnitude than those in a cascade representation. This can lead to slightly greater sensitivity in numerator terms compared to denominator terms.

Noise Behavior:

A unique characteristic of the parallel structure is the way round-off noise accumulates. Each branch introduces its own quantization noise due to finite-precision arithmetic. Unlike the cascade structure, where noise propagates sequentially, in the parallel structure the noise signals from all branches are combined directly at the output summer.

- If the noise sources generated in each branch are assumed to be uncorrelated, then the total noise power at the output is simply the sum of the noise powers contributed by each branch.
- This means that while the noise from each branch does not amplify through others, the overall noise level at the output can be higher because of the direct addition of all branch contributions.

- Careful scaling and allocation of dynamic range within each branch are necessary to balance precision and minimize the total output noise.

In summary, the parallel structure avoids sequential amplification of noise but accumulates the noise contributions additively at the final summation.

Conclusion

Mastering finite-word-length effects is an essential discipline in DSP engineering. It requires a synergistic understanding of signal theory, numerical analysis, and computer architecture to transform a perfect mathematical model into a reliable and efficient physical realization.

Part D:

Theory

In practical digital filter implementations, coefficients and internal signals cannot be represented with infinite precision due to hardware limitations. These finite-precision effects introduce quantization errors, which can alter the designed filter's behavior. The impact of quantization depends strongly on the filter structure — particularly on how arithmetic operations and feedback paths are arranged.

1. Quantization in IIR Filters

An Infinite Impulse Response (IIR) filter uses both present and past input as well as past output values, typically expressed as:

$$y(n) = -\sum_{k=1}^N a_k y(n-k) + \sum_{k=0}^M b_k x(n-k) \quad (1)$$

where $x(n)$ is the input signal, $y(n)$ is the output signal, a_k and b_k are the filter coefficients, and N and M are the orders of the recursive and feedforward parts respectively.

Finite word length effects arise mainly due to:

- **Coefficient Quantization:** Filter coefficients are approximated by quantized values, which can shift pole-zero locations and affect frequency response.
- **Product Quantization:** Multiplication results are quantized before accumulation.
- **Round-off Noise:** Each arithmetic operation introduces small errors that accumulate in feedback loops, possibly causing limit cycles.

These effects may result in magnitude and phase distortion, reduced stability, or self-sustained oscillations in the output.

2. Direct Form Realization

Structure:

- **Direct Form-I (DF-I):** Uses separate delay lines for input and output, implementing the full difference equation directly.
- **Direct Form-II (DF-II):** More memory-efficient, sharing delay elements for input and output.

Quantization Effects:

- Poles are highly sensitive to coefficient rounding, affecting all computations simultaneously.
- Round-off errors in feedback paths accumulate, potentially causing overflow or limit cycles.
- DF-II amplifies round-off errors more than DF-I due to concentrated feedback paths.

Practical Implication: Direct forms are suitable mainly for low-order filters; high-order filters require careful scaling to maintain stability.

3. Cascade Form Realization

Structure: High-order filters are realized as a series of second-order (biquad) sections:

$$H(z) = \prod_{i=1}^K H_i(z) \quad (2)$$

Each section handles part of the filter's poles and zeros.

Quantization Effects:

- Coefficient quantization is localized to each biquad, resulting in smaller pole shifts.
- Round-off noise in one section is attenuated by subsequent sections.
- Proper section ordering (e.g., placing low-gain sections first) minimizes noise amplification.

Advantages: Cascade form is robust to quantization, stable for fixed-point implementation, and allows individual section scaling.

4. Series (Parallel) Form Realization

Structure: The filter is represented as a sum of simpler sections, often via partial fraction decomposition:

$$H(z) = \sum_{i=1}^K H_i(z) \quad (3)$$

Each section operates independently in parallel, and the outputs are summed.

Quantization Effects:

- Coefficient quantization affects each branch locally, reducing the risk of instability.
- Minimal feedback interaction between branches ensures higher numerical stability.
- Summation may introduce minor amplitude errors.

Advantages: Series (parallel) form is highly stable, robust to coefficient errors, and suitable for high-order filters, though hardware complexity is higher.

Structure	Coefficient Sensitivity	Round-off Noise	Stability	Hardware
Direct Form	High	High	Poor	Low
Cascade Form	Low (localized errors)	Moderate	Good	Moderate
Series Form	Low (independent sections)	Low	Very Good	High

Table 13: Comparison of Quantization Effects in Different IIR Filter Structures

```

1  klc;
2  clear;
3  close all;
4  [numerator,denominator] = ellip(7,2,30,0.4);
5  figure(1);
6  freqz(numerator,denominator);
7  denominator_roots = roots(flip(denominator));
8  figure(2);
9  polarplot(angle(denominator_roots),abs(denominator_roots),"o");
10 word_length = 12;
11 fraction_length = 8;
12 quantized_numerator = quantiznumeric(numerator,1,word_length,fraction_length);
13 quantized_denominator = quantiznumeric(denominator,1,word_length,fraction_length);
14 figure(3);
15 freqz(quantized_numerator,quantized_denominator);
16 quantized_denominator_roots = roots(flip(quantized_denominator));
17 figure(4);
18 polarplot(angle(quantized_denominator_roots),abs(quantized_denominator_roots),"o");

```

Figure 43: Code Part 1, for Direct, Wordlength - N

```

[numerator,denominator] = ellip(7,2,30,0.4);

figure(1);
freqz(numerator,denominator);

random = [1.2,2.5,3,4.743,5.2223,6,7,8];
word_length = 12;
fraction_length = 8;

random_transfer_function_1 = [random,denominator];
random_transfer_function_2 = [numerator,random];

casc_num_1 = quantizenumeric(numerator,1,word_length,fraction_length);
casc_den_1 = quantizenumeric(denominator,1,word_length,fraction_length);

casc_num_2 = quantizenumeric(numerator,1,word_length,fraction_length);
casc_den_2 = quantizenumeric(denominator,1,word_length,fraction_length);

final_num = conv(casc_num_1,casc_num_2);
final_den = conv(casc_den_1,casc_den_2);

figure(2);
freqz(final_num,final_den);

k = roots(flip(final_den));

figure(3);
polarplot(angle(k),abs(k),"o");

```

Figure 44: Code Part 2, for cascade, Wordlength - N

```

clc;
clear;
close all;
[numerator,denominator] = ellip(7,2,30,0.4);
figure(1);
freqz(numerator,denominator);
denominator_roots = roots(flip(denominator));
figure(2);
polarplot(angle(denominator_roots),abs(denominator_roots),"o");
[res,poles,k] = residue(numerator,denominator);
word_length = 12;
fraction_length = 8;
poles_quantized = quantizenumeric(poles,1,word_length,fraction_length);
figure(3);
polarplot(-angle(poles_quantized),abs(poles_quantized).^-1,"o");

```

Figure 45: Code Part 3, for parallel, Wordlength - N

Plots

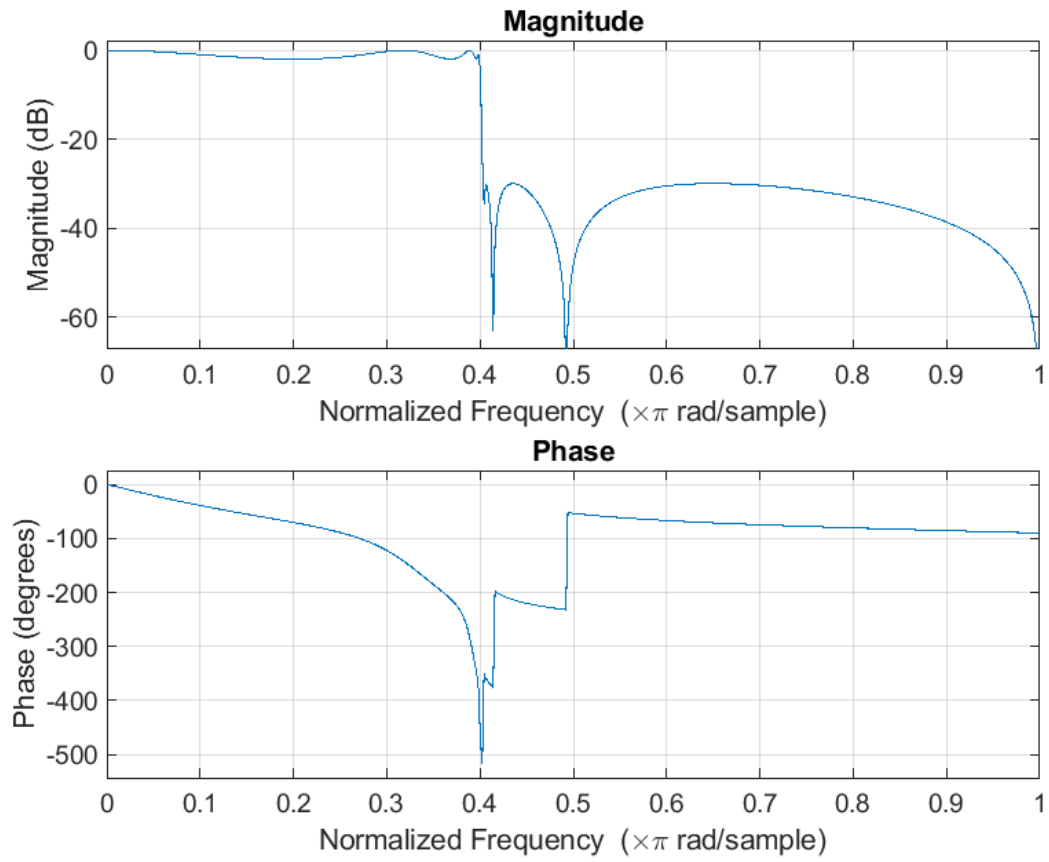


Figure 46: Transfer function of Elliptical Filter for N=8

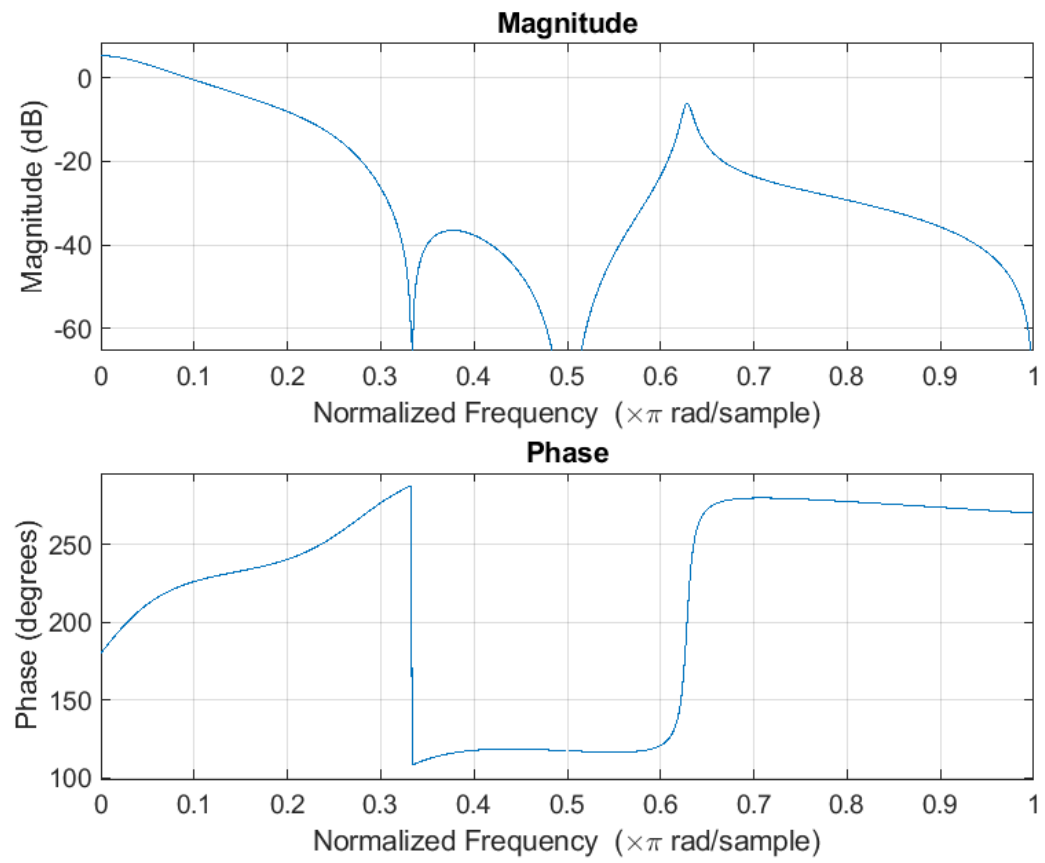


Figure 47: Numerator and Denominator Individually Quantized for $N=8$

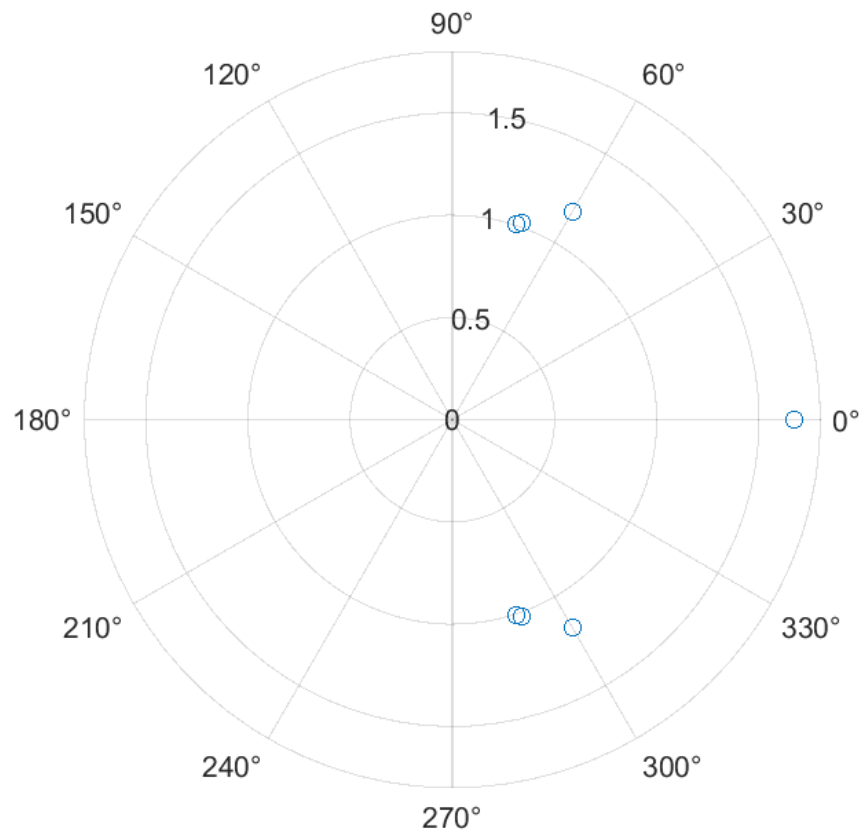


Figure 48: Polar Plot of Transfer Function for $N=8$

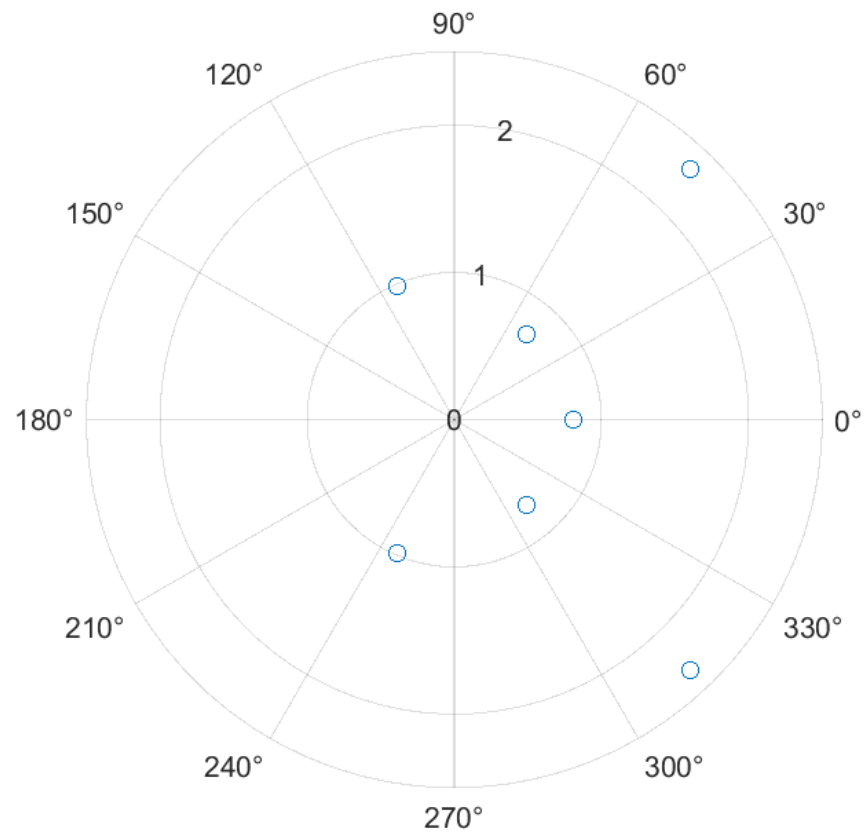


Figure 49: Polar Plot of Quantized Transfer Function for $N=8$

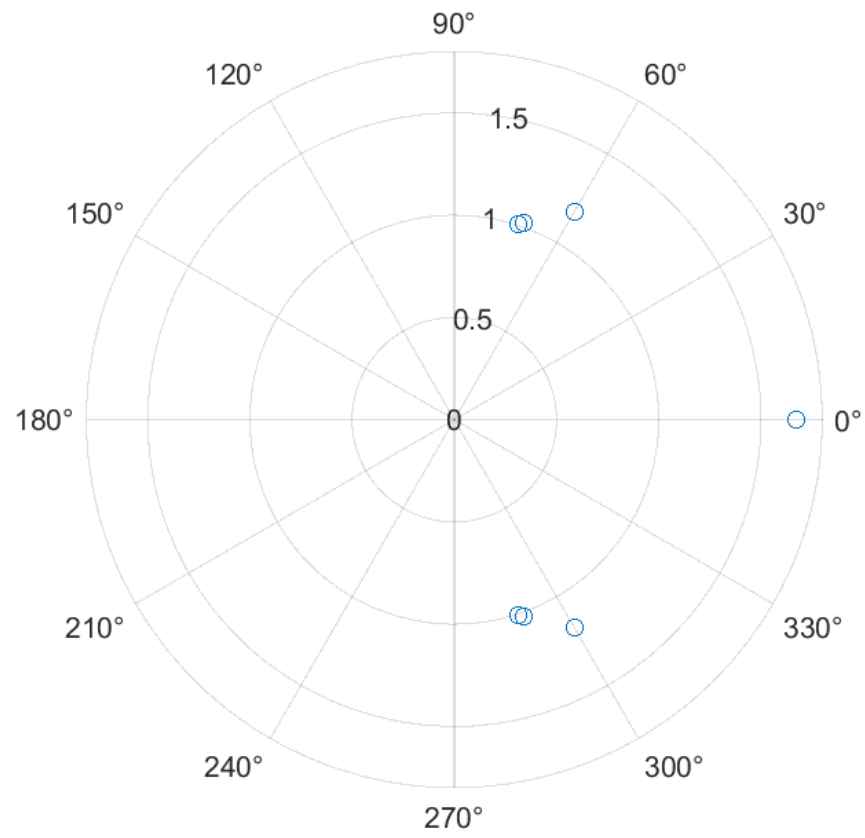


Figure 50: PARALLEL: Polar Plot of Transfer Function (Residues) for $N=8$

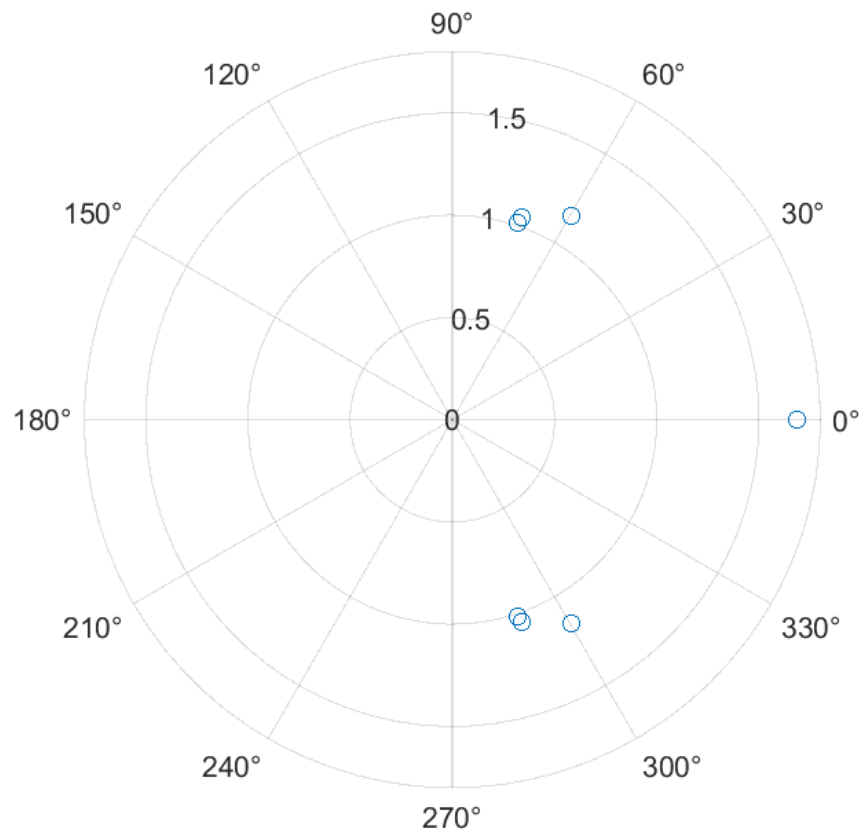


Figure 51: PARALLEL: Polar Plot of Quantized Transfer Function (Residues) for $N=8$

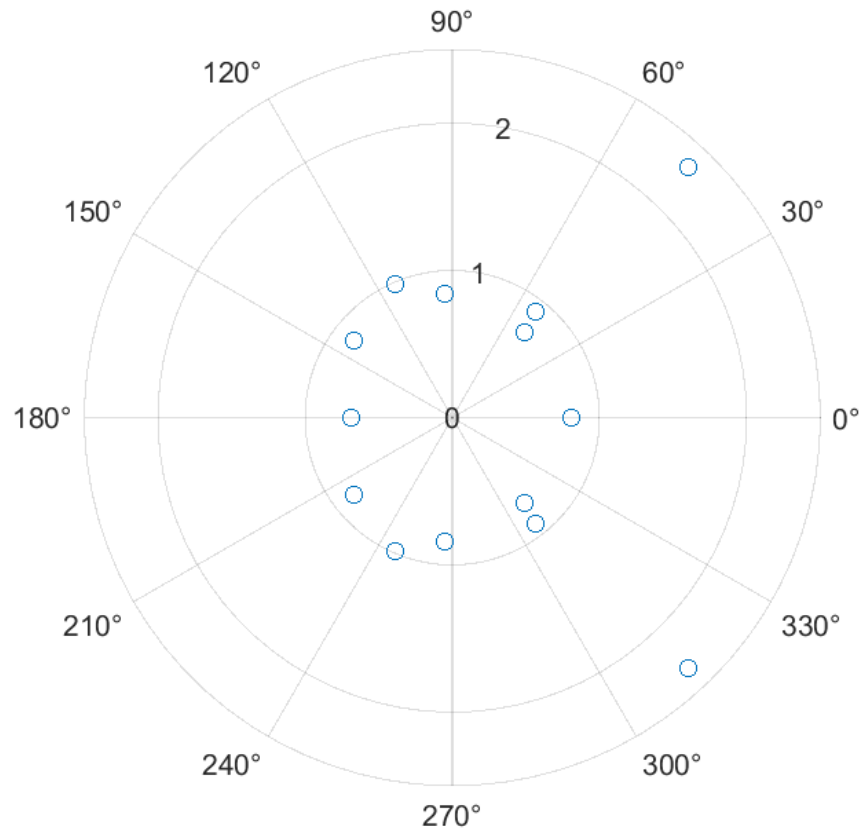


Figure 52: CASCADED: Polar Plot of Quantized Transfer Function for $N=8$

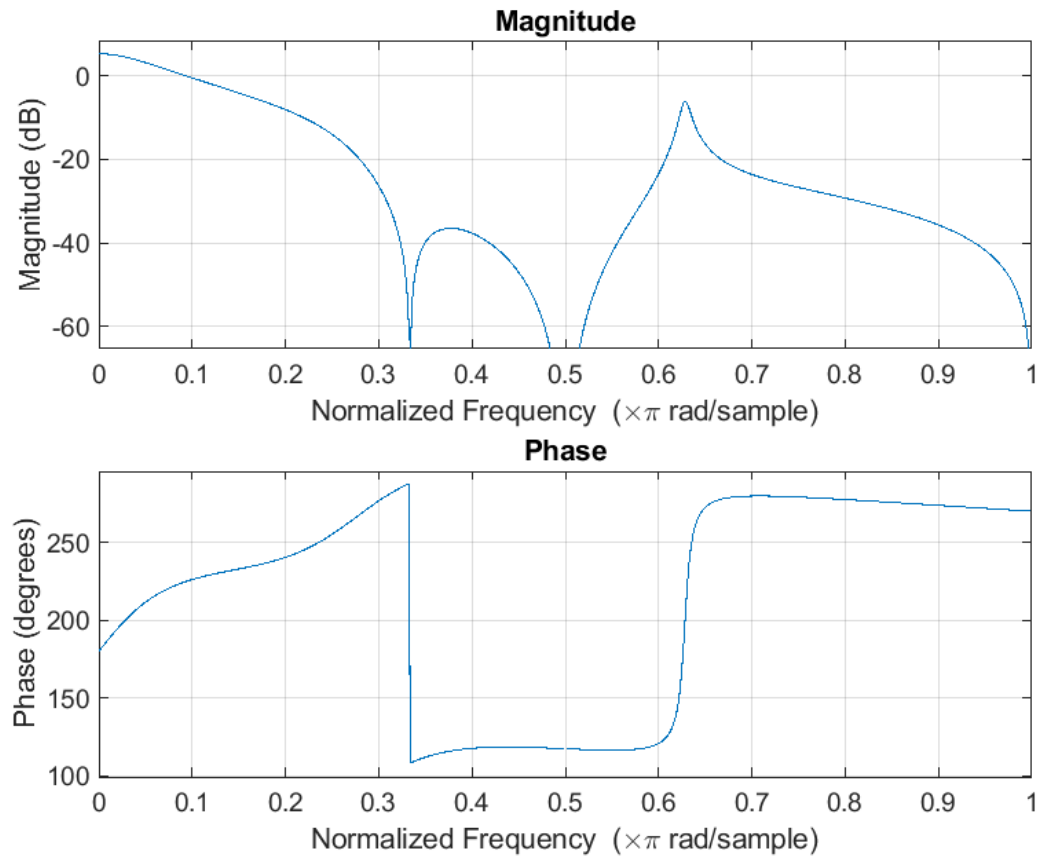


Figure 53: CASCADED: Frequency Response of Transfer Function for N=8

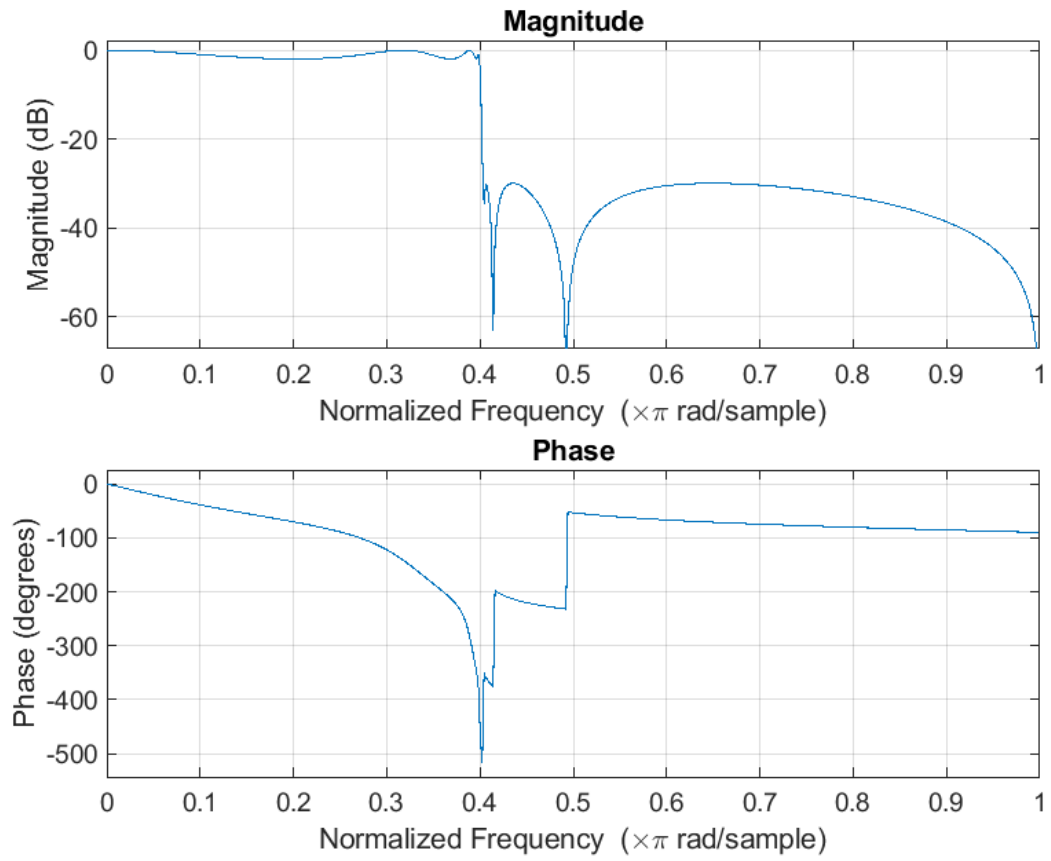


Figure 54: Transfer function of Elliptical Filter for N=6

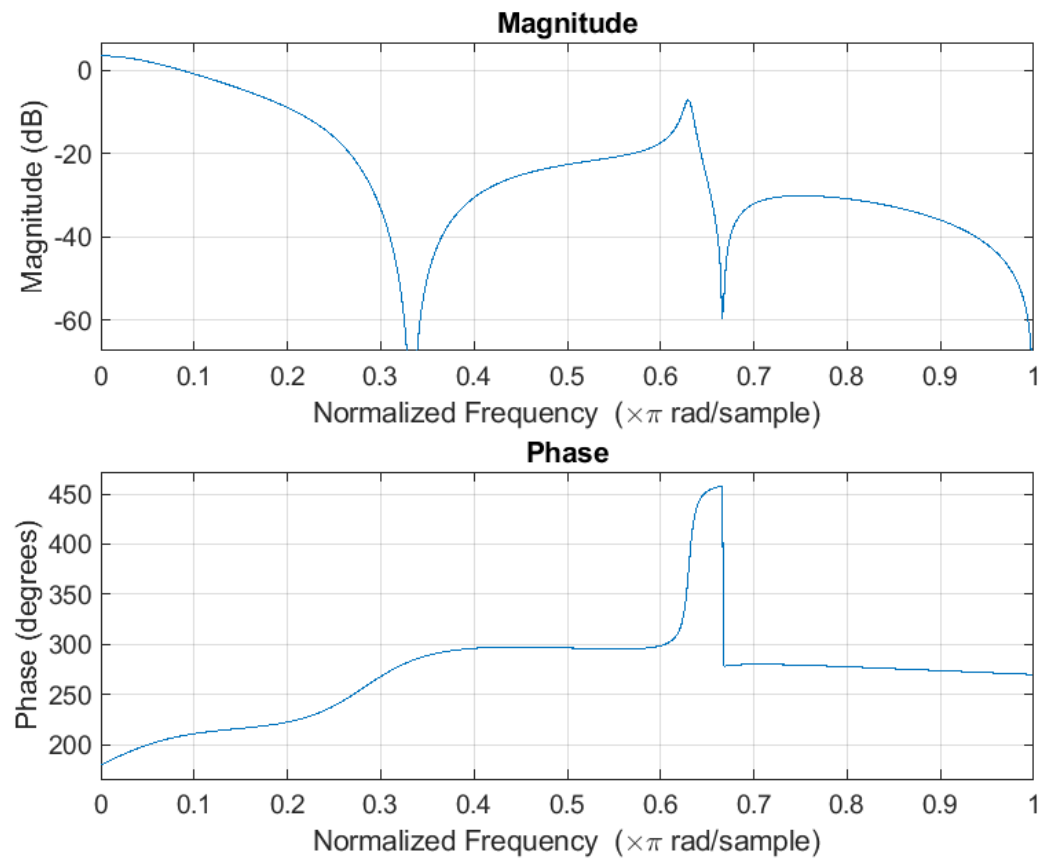


Figure 55: Numerator and Denominator Individually Quantized for $N=6$

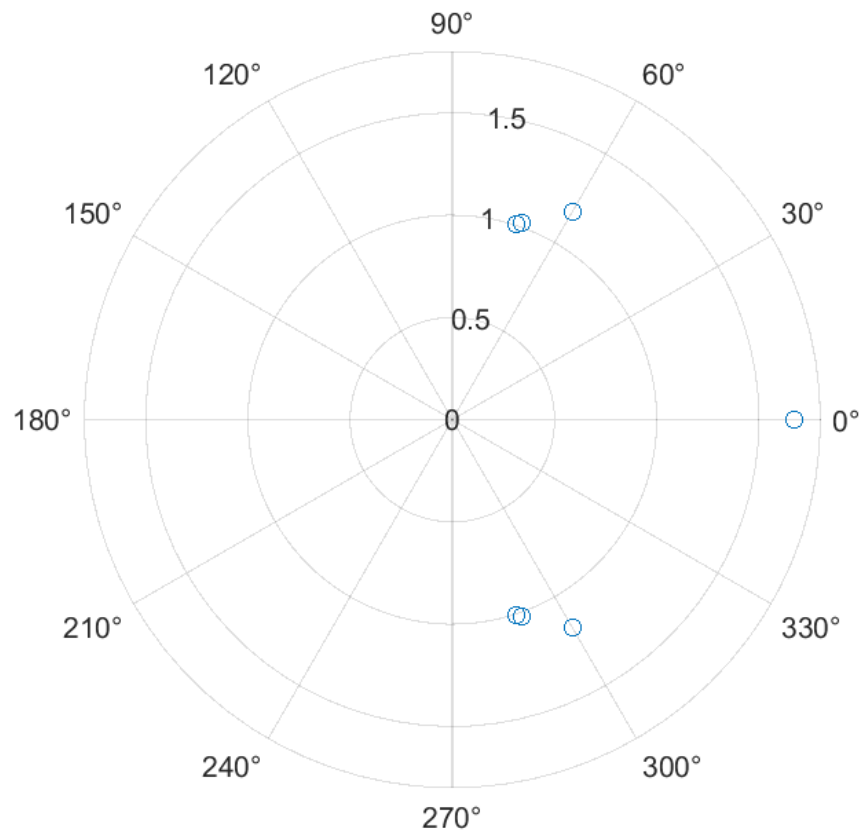


Figure 56: Polar Plot of Transfer Function for $N=6$

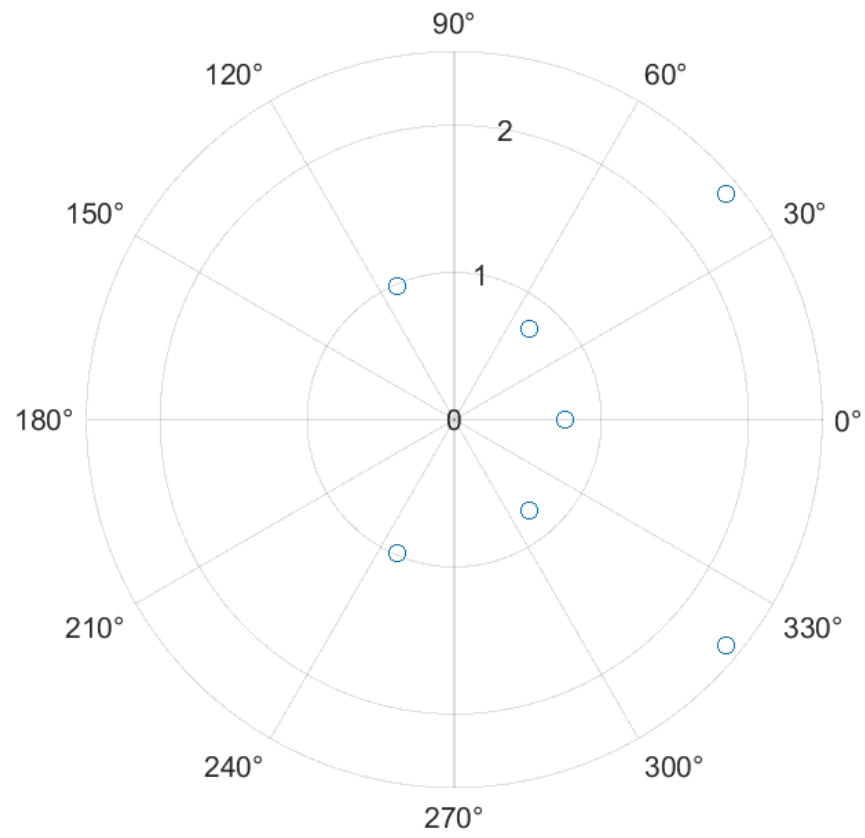


Figure 57: Polar Plot of Quantized Transfer Function for $N=6$

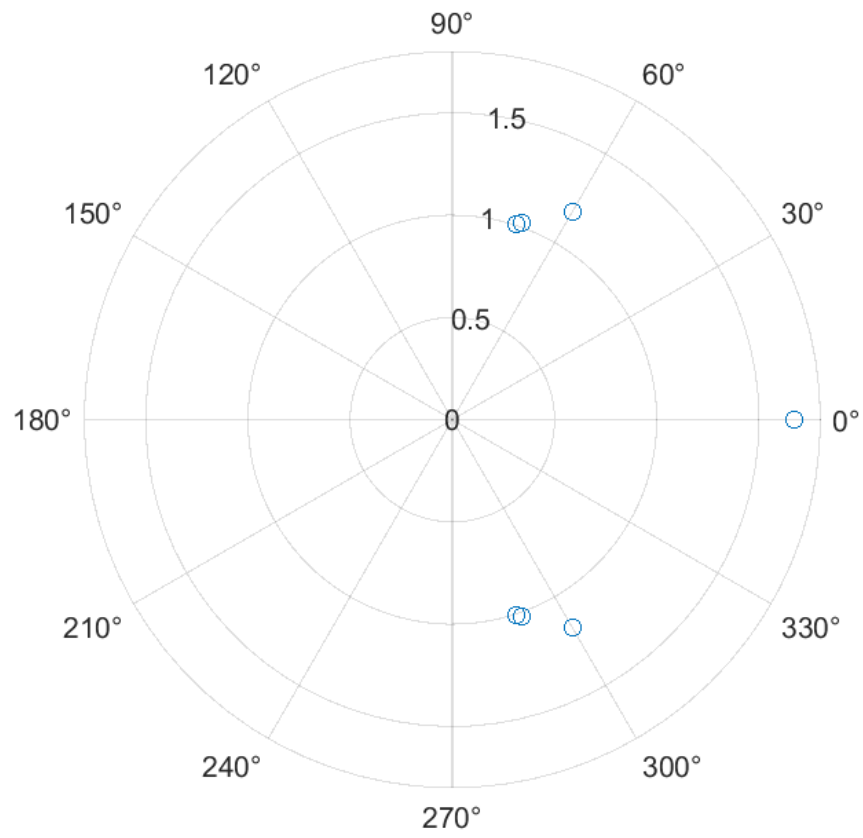


Figure 58: PARALLEL: Polar Plot of Transfer Function (Residues) for $N=6$

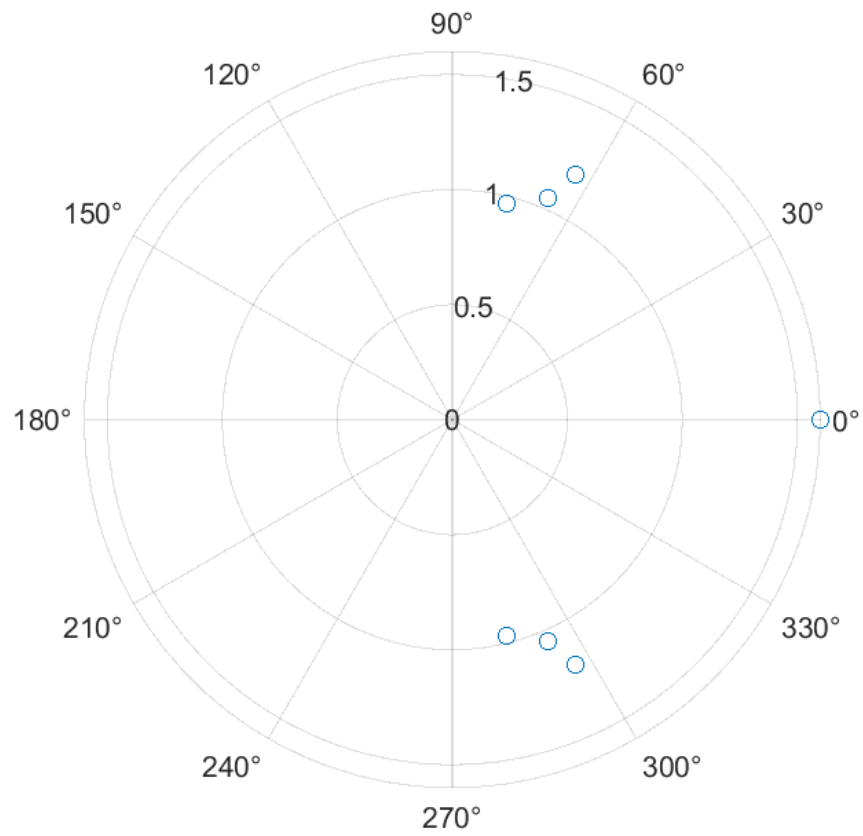


Figure 59: PARALLEL: Polar Plot of Quantized Transfer Function (Residues) for $N=6$

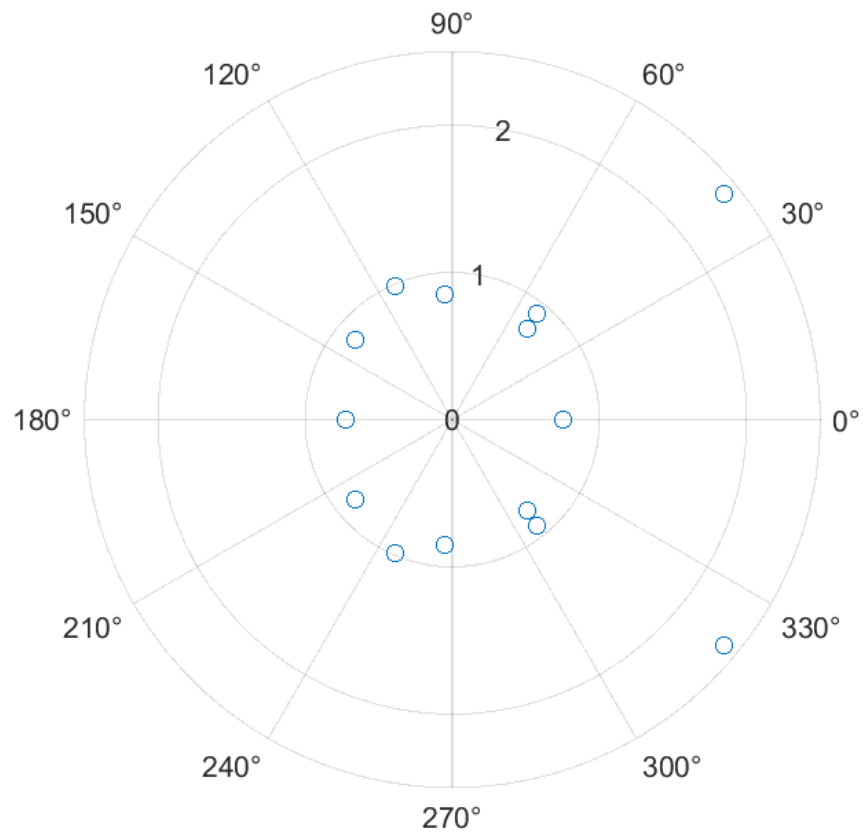


Figure 60: CASCADED: Polar Plot of Quantized Transfer Function for $N=6$

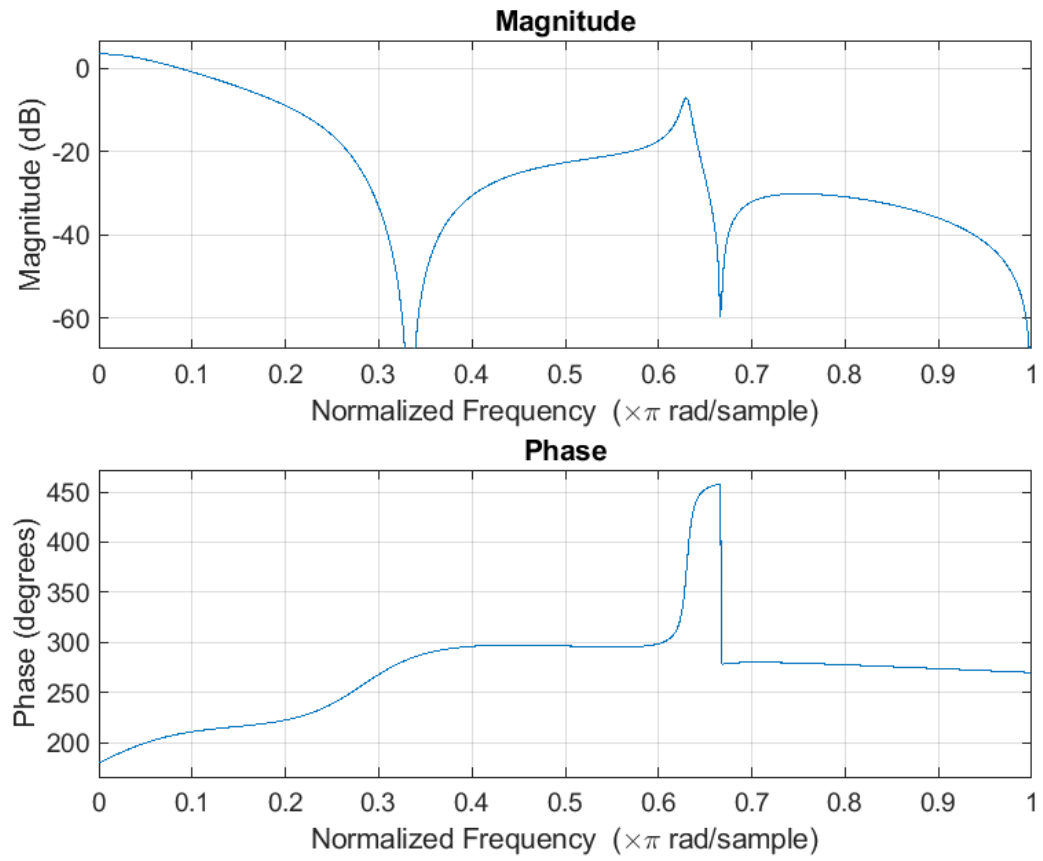


Figure 61: CASCADED: Frequency Response of Transfer Function for N=6

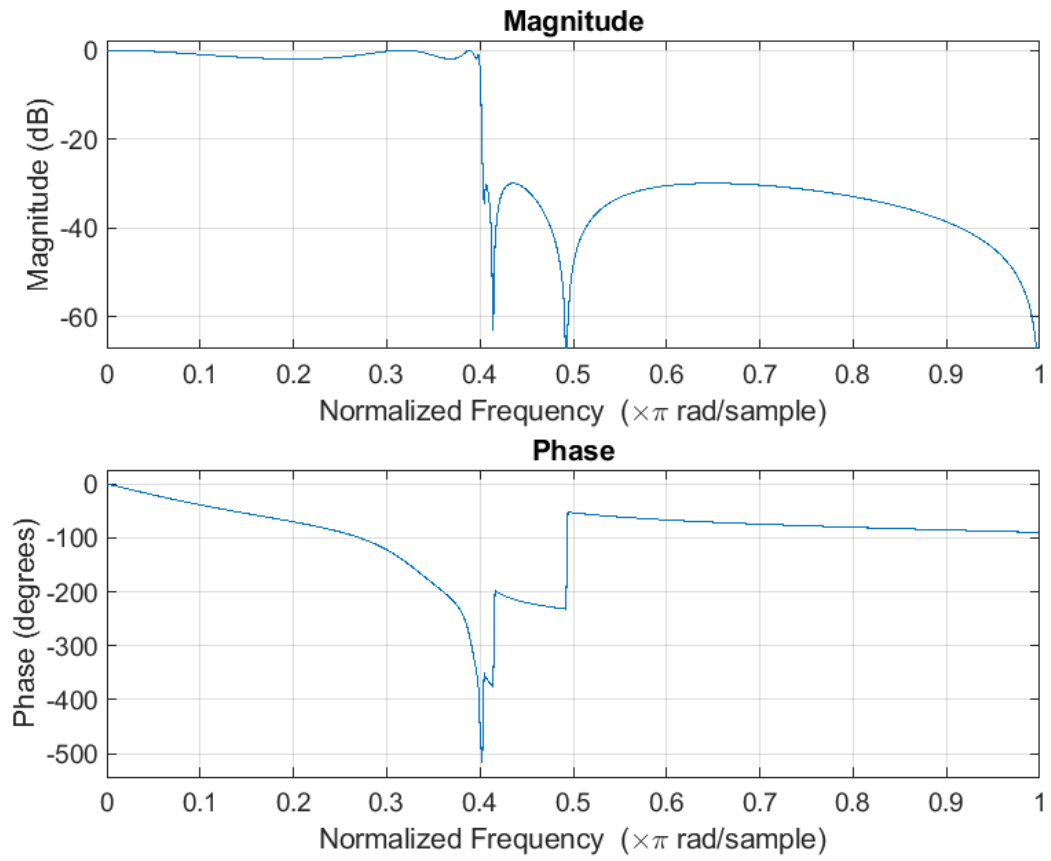


Figure 62: Transfer function of Elliptical Filter for N=4

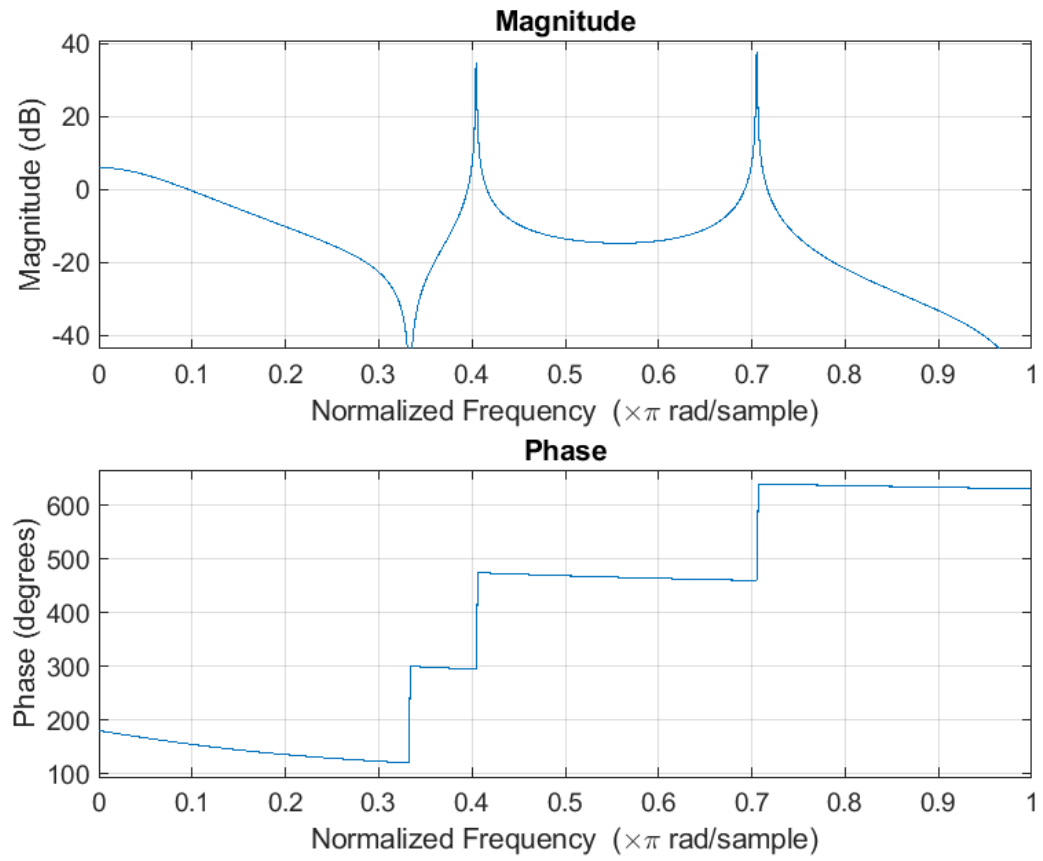


Figure 63: Numerator and Denominator Individually Quantized for $N=4$

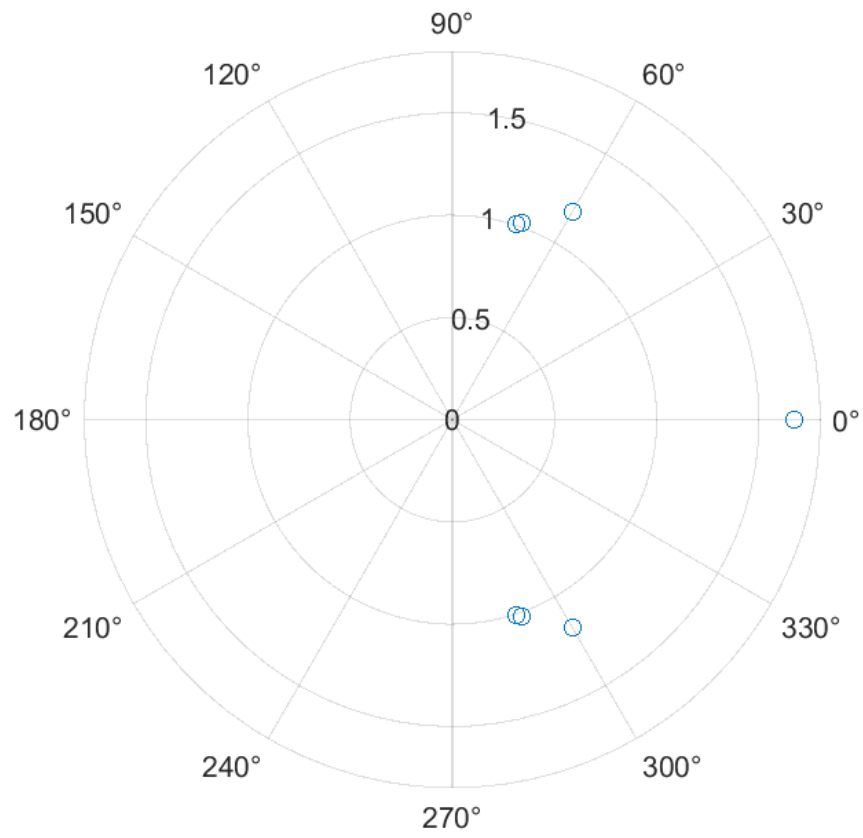


Figure 64: Polar Plot of Transfer Function for $N=4$

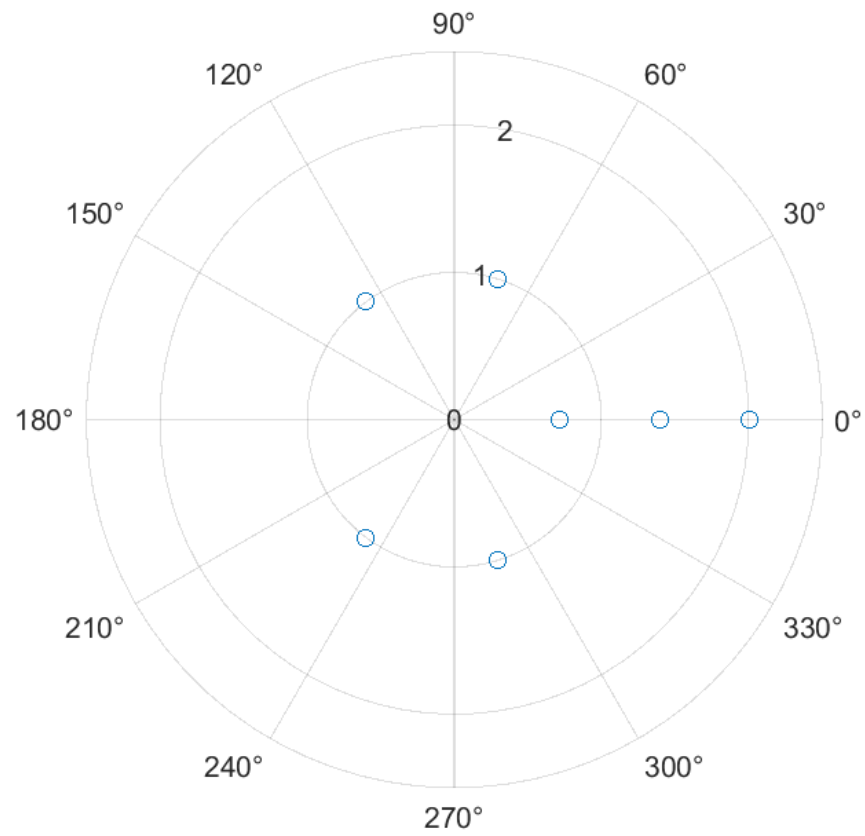


Figure 65: Polar Plot of Quantized Transfer Function for $N=4$

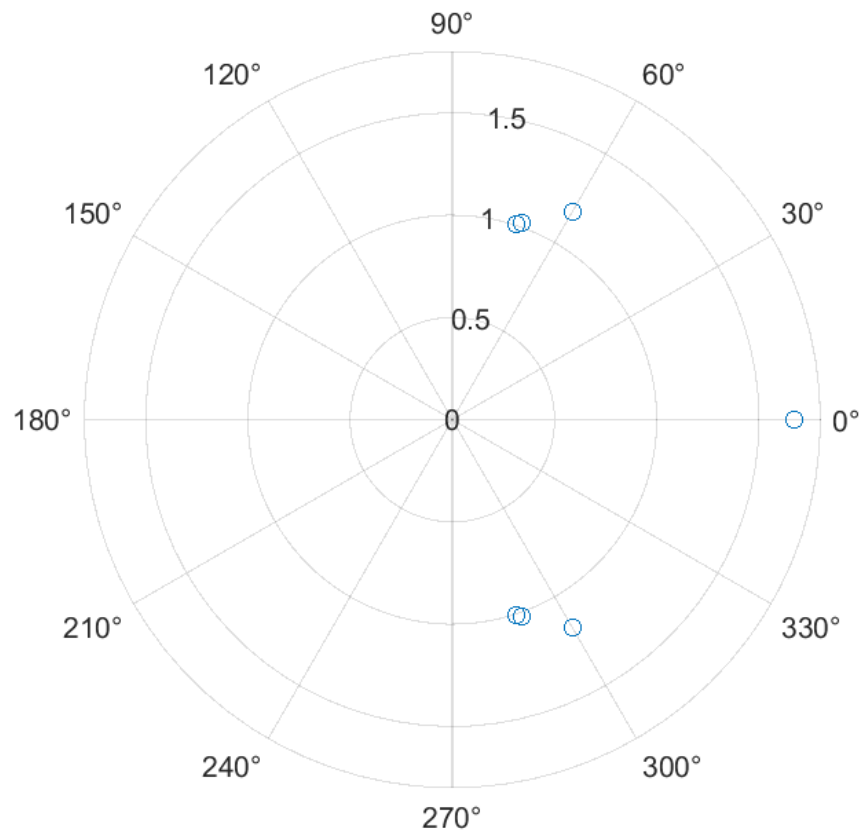


Figure 66: PARALLEL: Polar Plot of Transfer Function (Residues) for $N=4$

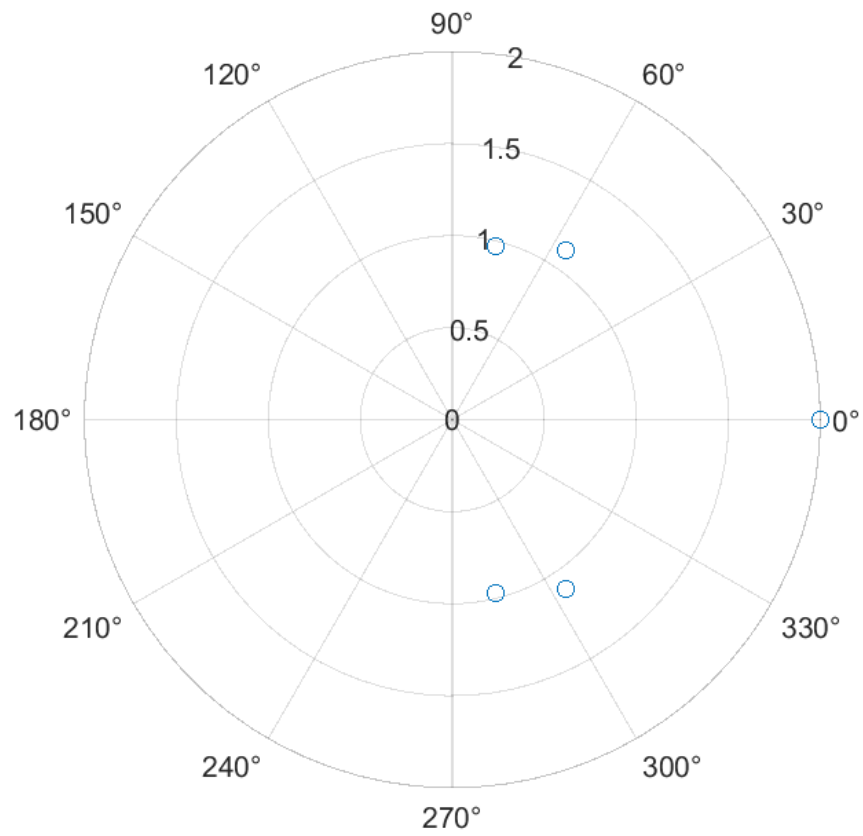


Figure 67: PARALLEL: Polar Plot of Quantized Transfer Function (Residues) for $N=4$

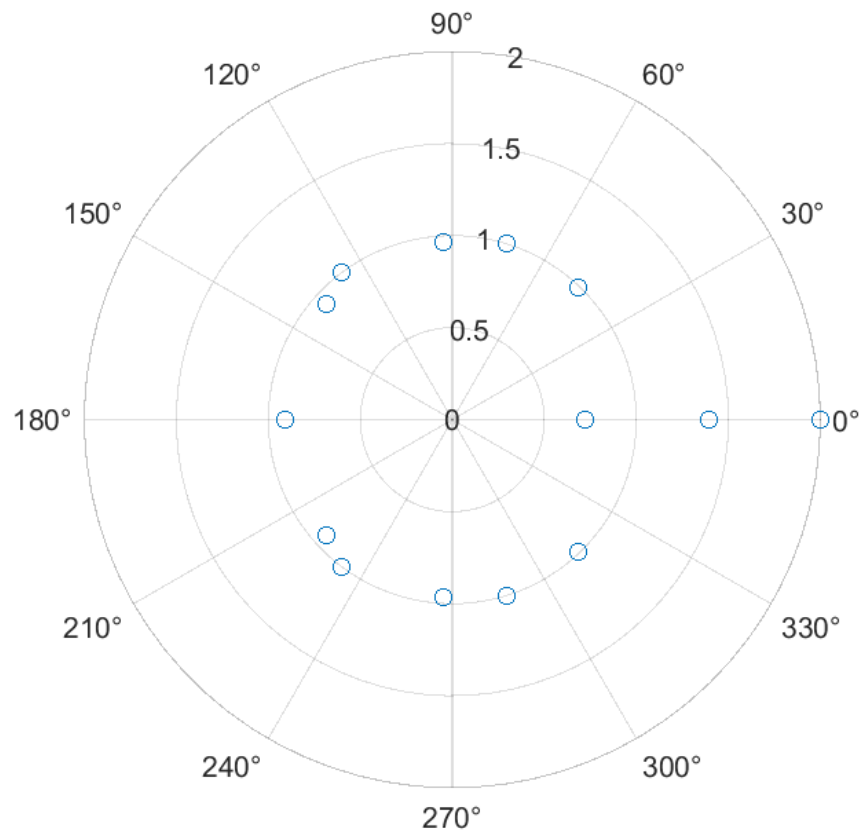


Figure 68: CASCADED: Polar Plot of Quantized Transfer Function for $N=4$

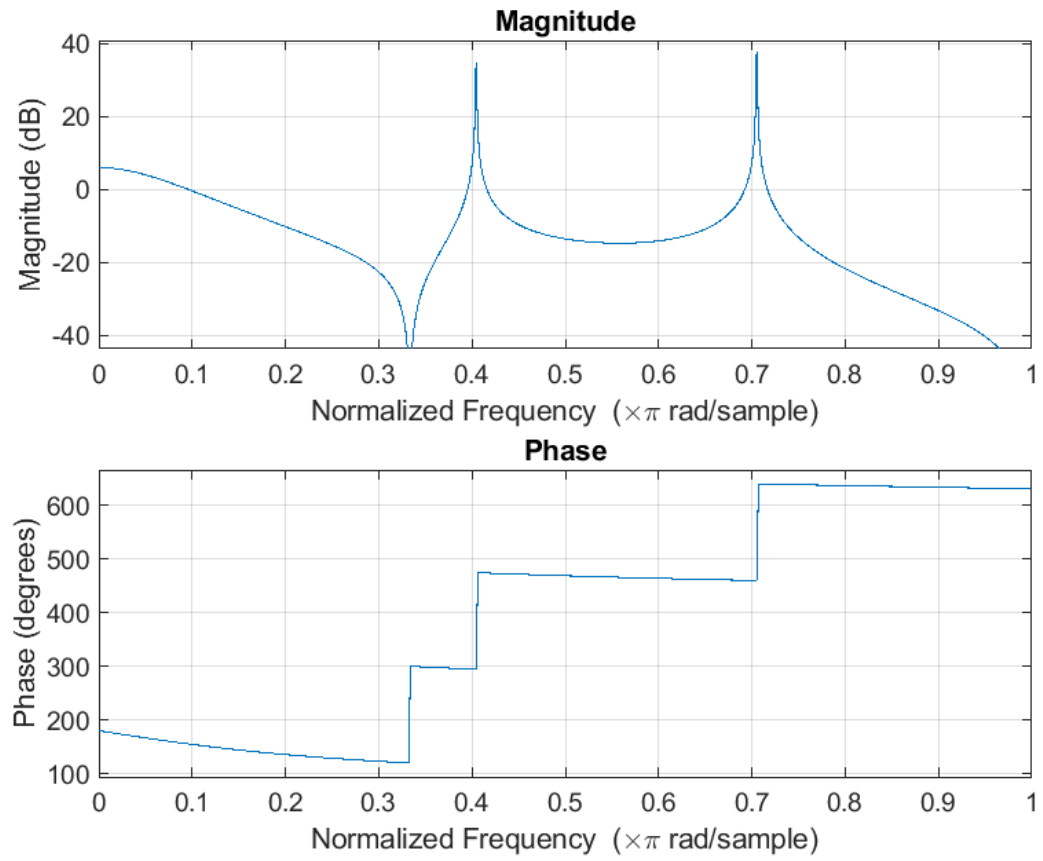


Figure 69: CASCADED: Frequency Response of Transfer Function for N=4

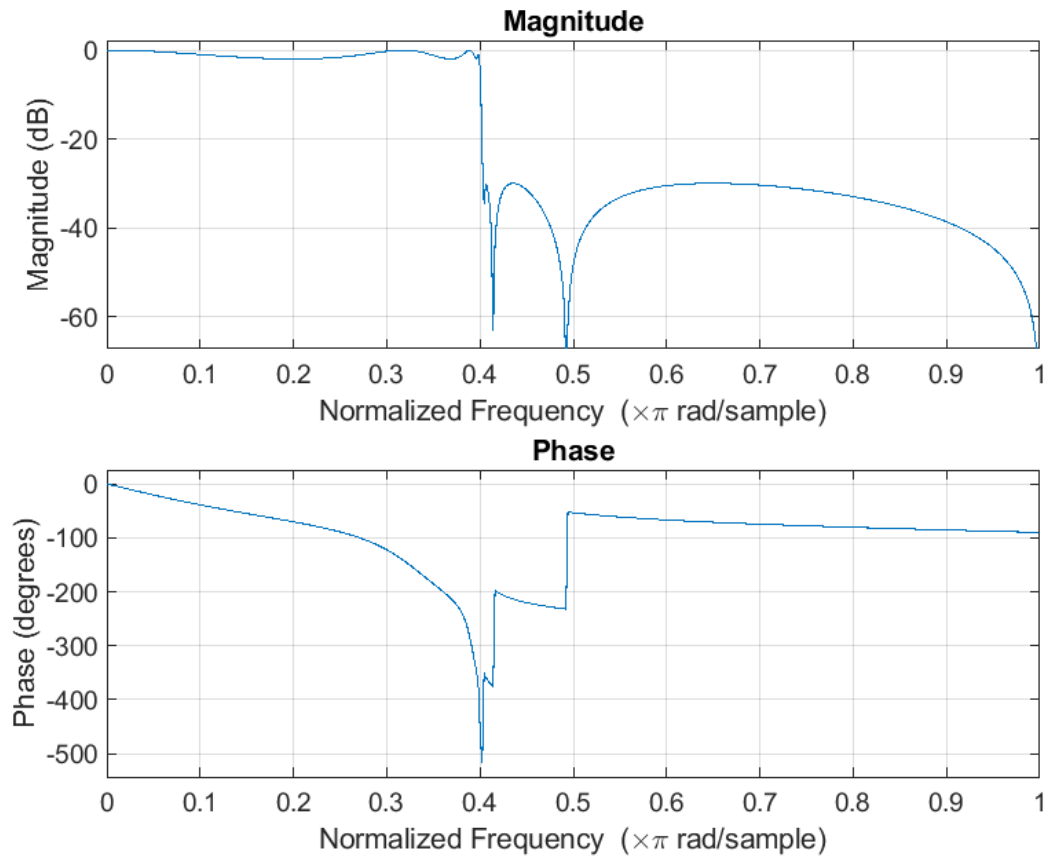


Figure 70: Transfer function of Elliptical Filter for $N=2$

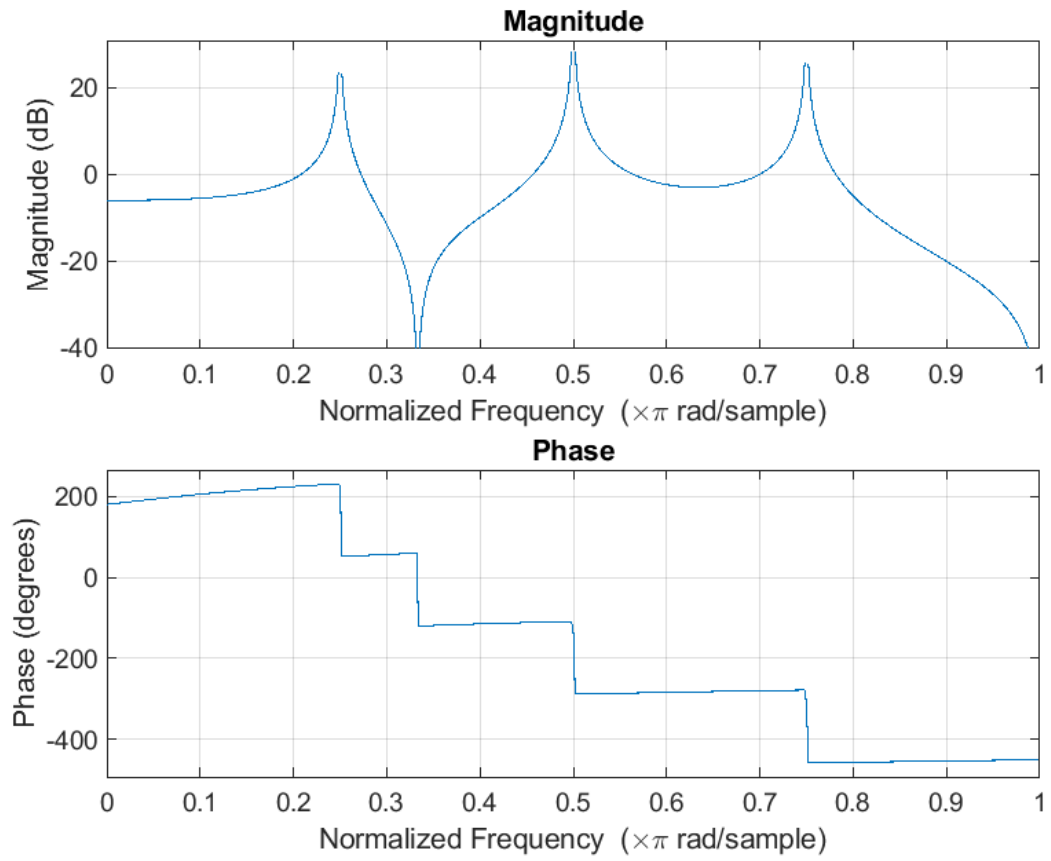


Figure 71: Numerator and Denominator Individually Quantized for $N=2$

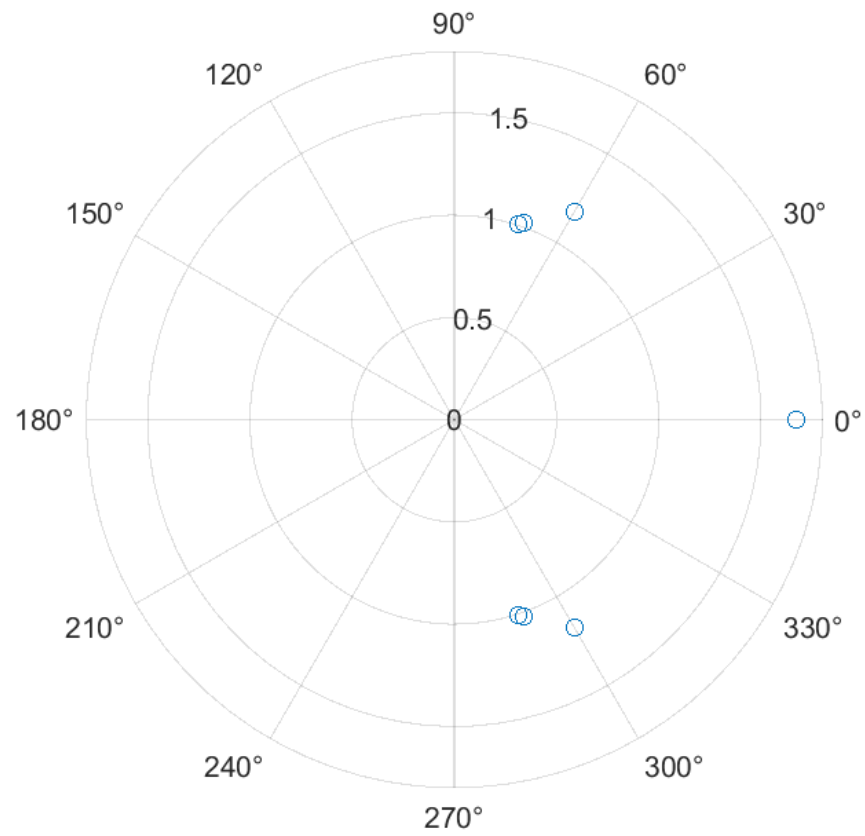


Figure 72: Polar Plot of Transfer Function for $N=2$

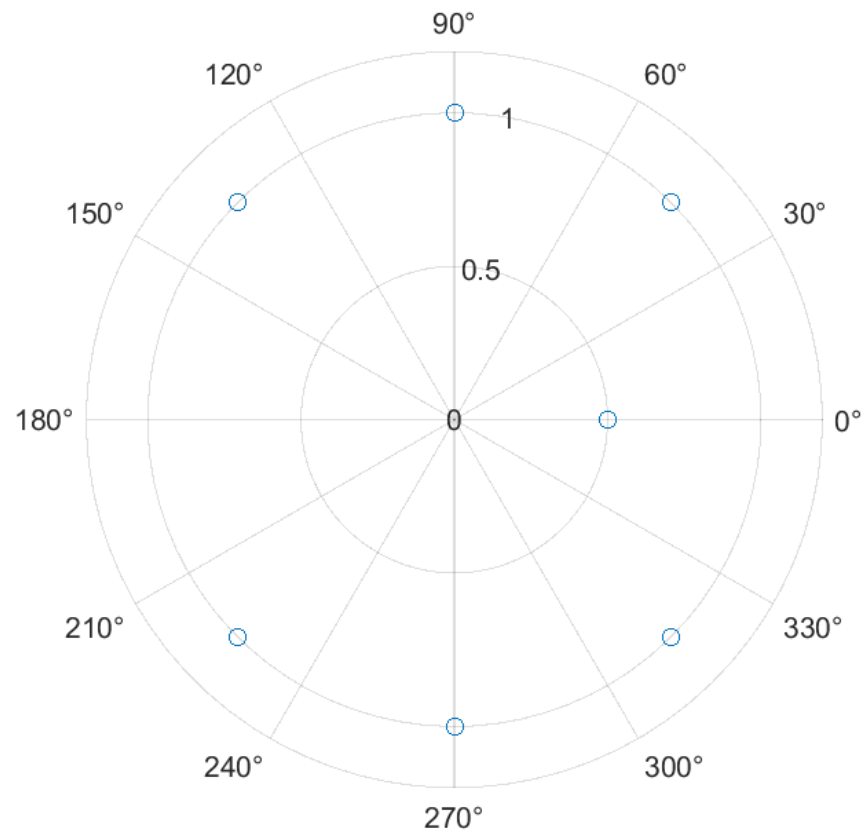


Figure 73: Polar Plot of Quantized Transfer Function for $N=2$

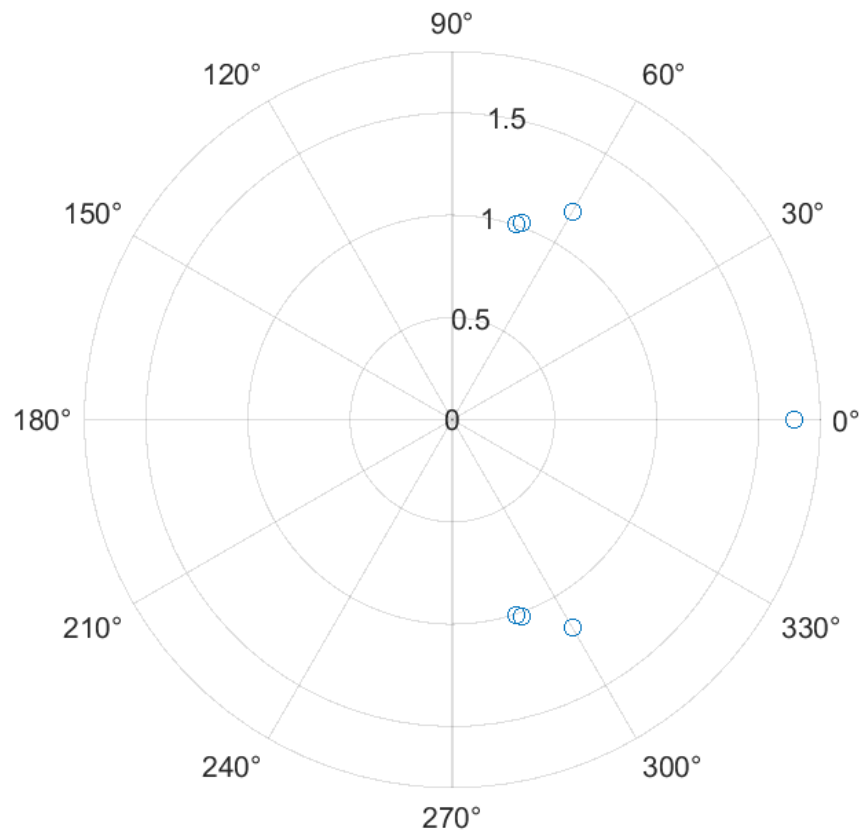


Figure 74: PARALLEL: Polar Plot of Transfer Function (Residues) for $N=2$

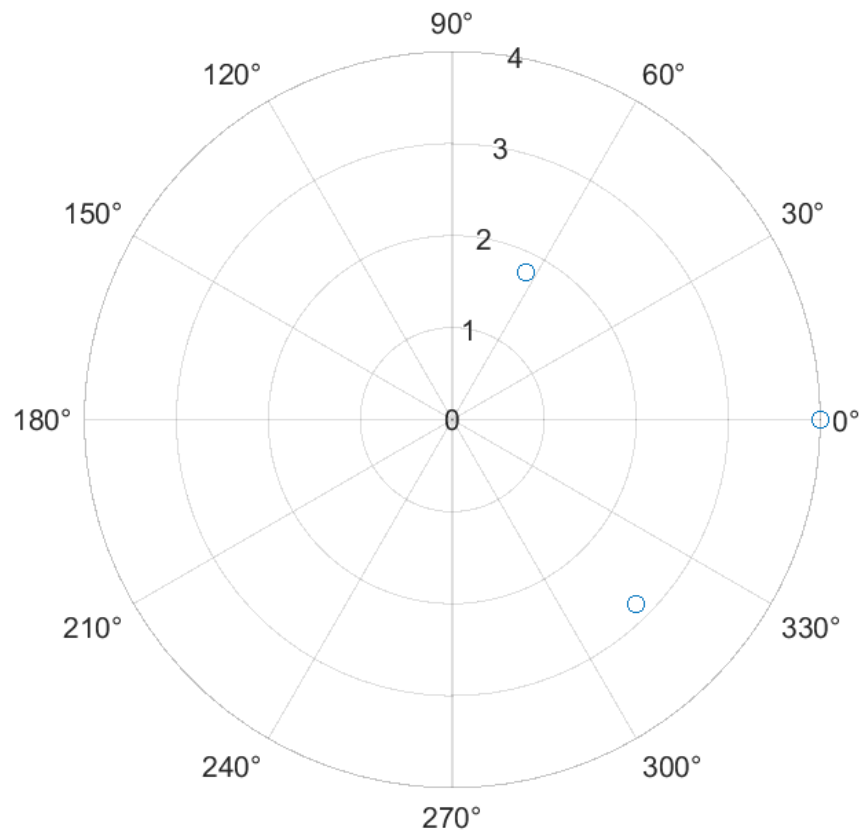


Figure 75: PARALLEL: Polar Plot of Quantized Transfer Function (Residues) for $N=2$

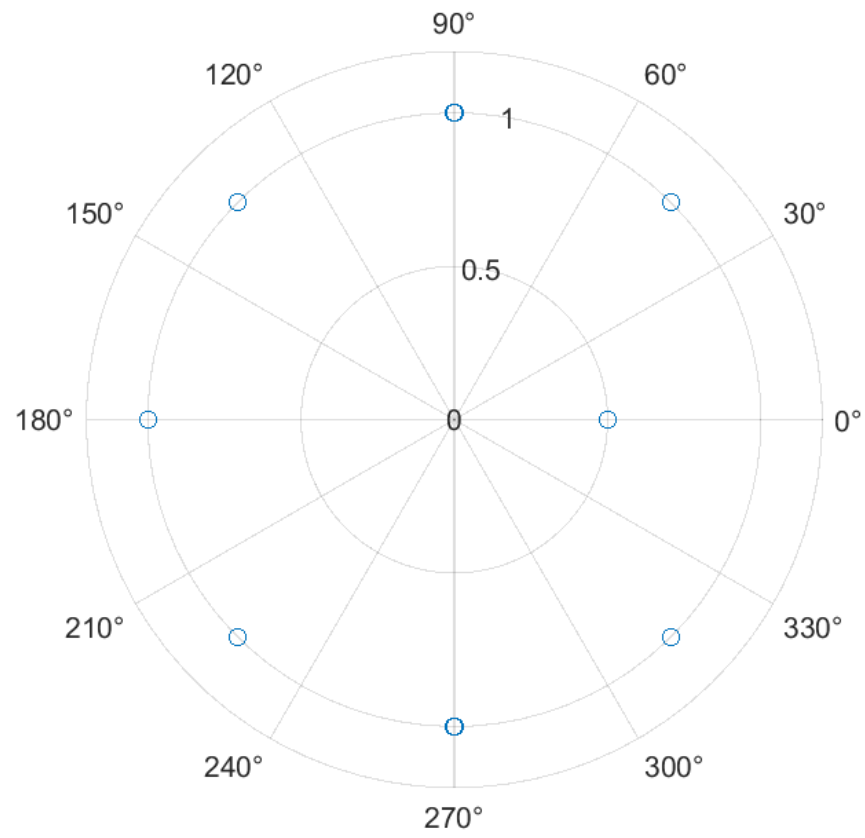


Figure 76: CASCADED: Polar Plot of Quantized Transfer Function for $N=2$

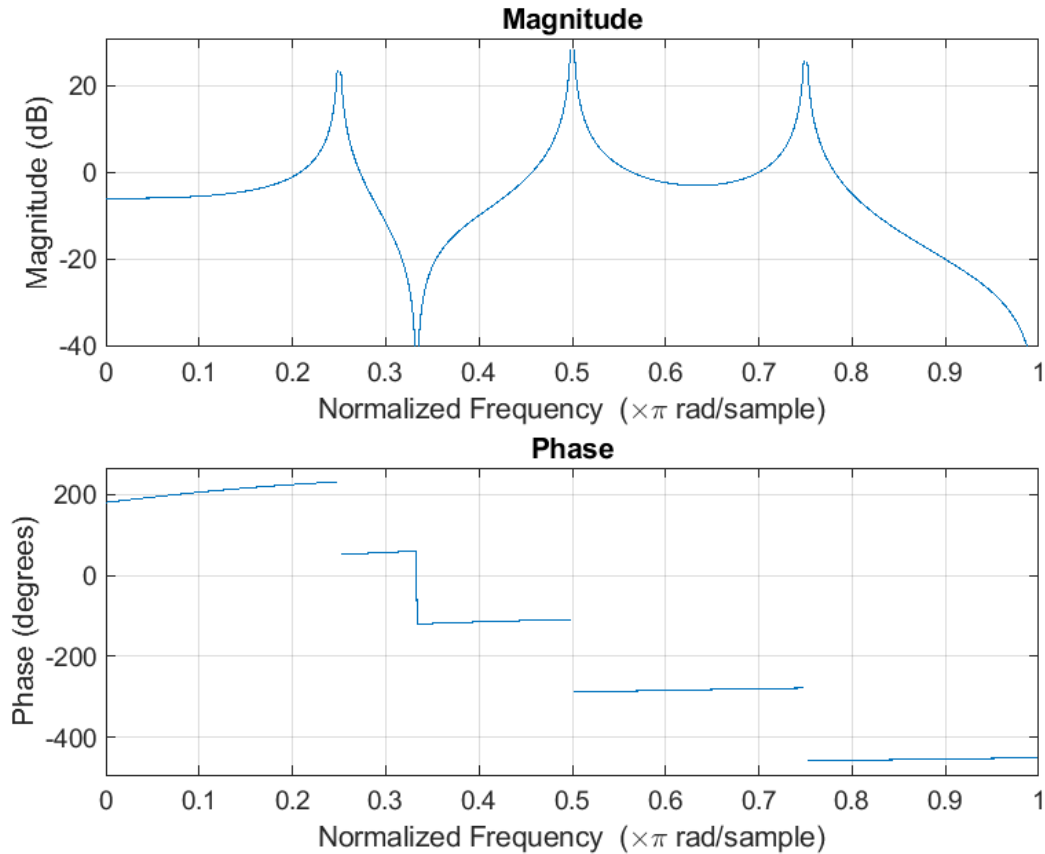


Figure 77: CASCADED: Frequency Response of Transfer Function for $N=2$

Overall Discussion

In this experiment, we focused on the basics of Digital Filter Design. First, we looked at the simplest technique, which is Windowing, where we appropriately truncate the ideal impulse response using a window function while reducing distortion in the Frequency Spectrum.

Windowing is often the simplest and it is very easy to understand. Filter Responses of Ideal Filters like Low Pass Filter, have sharp edges, which are in practice not realizable. Also the impulse response length is infinite and would theoretically require an infinite number of computations to compute the output signal. Also these filter responses are not causal, hence we cannot do real time Signal Processing on a Machine.

Windowing simply truncates the impulse response. We unknowingly use this filter design the most as simply truncating the filter response for an approximate output is simply multiplying the impulse response by the rectangular window. This offers a lot of disadvantages as there is multiplication in the time domain, which is convolution in the frequency domain. Due to this, the Ideal Filter Response get convolved with a Sinc Kernel, which causes "Ripples" both in the pass band and the stop band. Using Other Windows we can successfully mitigate this effect, with the trade off of using a larger number of points.

The Parks-McClellan (PM) and Least Squares algorithms are widely used in FIR filter design, each optimized for different design philosophies and application requirements. The PM algorithm uses the Remez exchange algorithm and Chebyshev approximation

theory to produce equiripple designs, minimizing the maximum error between the desired and actual frequency response in specified bands. This focus on the worst-case error leads to filters with controlled ripple and sharp transitions, making PM the methodology of choice for applications like communication and radar systems where strict adherence to passband and stopband specifications is essential.

By contrast, the Least Squares algorithm minimizes the total squared error between the desired and actual frequency response, yielding filters that provide the best average fit across the frequency bands. This solution typically results in less uniform ripples, but a lower overall mean error across the entire filter response. Least Squares techniques are especially advantageous in applications where global fidelity to the ideal response is more important than controlling the worst-case deviation, such as audio or image processing. Extensions like Constrained Least Squares (CLS) further enhance the approach by enabling designers to specify upper and lower bounds on the frequency response, allowing for flexible tradeoffs between transition width and ripple magnitude. Quantization in IIR filters arises due to the finite word length used to represent filter coefficients and perform arithmetic operations in digital hardware. Since IIR filters use feedback, even small quantization errors in coefficients or calculations can significantly affect the filter's behavior. These errors can shift the filter poles, potentially leading to instability or deviations in the frequency response from the intended design. Unlike FIR filters, the recursive nature of IIR filters amplifies these errors, making careful quantization management crucial.

For the cascade form, a higher-order IIR filter is implemented as a series (cascade) of several second-order sections (biquads). Each biquad processes the signal sequentially, passing its output to the next stage. This approach helps localize quantization errors within each section and reduces their overall impact on filter stability and performance. Cascade structures are widely preferred because they make coefficient quantization more manageable and improve numerical robustness.

For the parallel form, the filter is expressed as a sum of simpler filter components, often first- or second-order sections. Each section independently processes the input signal, and their outputs are summed to produce the final result. Parallel structures can offer design flexibility and can simplify filter realization for certain types of responses, but they may be more sensitive to coefficient quantization errors since errors in any branch directly affect the output sum.